You said:

help me create an appliation that will install on my windows computer. thsi is a program we will build together, you will then provide me with an msi file that i can use to then install in my windows computer. I want this application to be able to connect to my local sql express instance so that it will save and retrive from the sql express instance. i want you to provide me with ideas of an application we can build you can provide an msi for and i can install locally in my windows 11 laptop. provide me with 5 ideas for a program we can build together. this will run localhost and if needed i have iis installed already so it can create application pools and a site if needed to run,

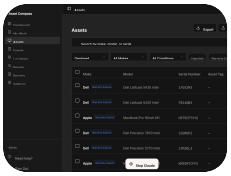
ChatGPT said:

I love this project idea! I think we can build something genuinely useful, not just a demonstration application. My goal would be to help you develop a **professional Windows 11 application using C#, .NET, and Microsoft SQL Server Express**, package it into an MSI installer, and eventually have a program you can install, uninstall, upgrade, and maintain like commercial software.

Since you already have SQL Server Express and IIS available, we have an excellent foundation for building a real-world application.

I would recommend using **C# with .NET 10 LTS**, SQL Server Express, and either Windows Forms or ASP.NET Core, depending on which application you select.

Five applications we could build together

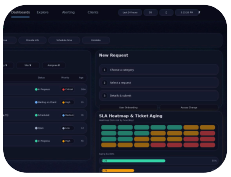


1. TovarTech IT Asset Manager My top choice

A Windows application to manage your computers, servers, virtual machines, software licenses, and hardware inventory.

Features: Add/edit/delete assets, track IP addresses, operating systems, serial numbers, warranty dates, asset status, and maintenance history. Search records and export reports.

SQL database: `TovarTechAssets`



2. TovarTech Help Desk & Ticketing System

Build a complete IT support application for managing incidents, technical problems, and customer requests.

Features: Create tickets, assign priorities, track troubleshooting steps, record resolutions, manage customers, attach screenshots, and generate support reports.

SQL database: `TovarTechHelpDesk`



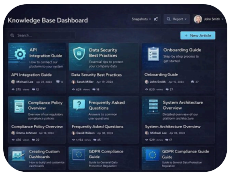
3. TovarTech Server Monitoring Center

Create a centralized monitoring application for your Windows, Linux, VMware ESXi, and Proxmox lab servers.

Features: Monitor server availability, ping responses, CPU and memory where supported, disk space, service health, and historical events. Display charts and configurable alerts.

SQL database: `TovarTechMonitoring`

This would be more advanced because remote metrics require appropriate authentication, agents, or management APIs.



4. TovarTech Knowledge Base Manager

Build a searchable repository for technical documentation, troubleshooting procedures, training lessons, and knowledge base articles.

Features: Create articles, organize categories, store commands and scripts, search technical solutions, attach files, and export documentation to PDF.

SQL database: `TovarTechKnowledgeBase`



5. TovarTech Project & Task Manager

Develop a Windows application for tracking personal projects, IT deployments, learning objectives, and recurring maintenance tasks.

Features: Create projects and tasks, set deadlines, manage progress, track time, record notes, and display project completion charts.

SQL database: `TovarTechProjects`

Comparing our five choices

Application	Complexity	Learning value
IT Asset Manager	Moderate	CRUD, SQL, reporting
Help Desk	Moderate	Relationships, workflows
Server Monitoring	Advanced	APIs, services, networking
Knowledge Base	Moderate	Search, files, SQL
Project Manager	Beginner–moderate	Forms, SQL, dashboards

All five can be built as locally installed Windows applications. IIS is optional for the desktop versions, but it becomes useful if we decide to provide browser-based access.

How I propose we build and install it

Proposed Windows desktop architecture

Windows 11 Desktop Application

C# / .NET 10 / Windows Forms

↓

Microsoft.Data.SqlClient

↓

SQL Server Express

Local instance • Application database • Tables

↓

Installer and configuration

↓

TovarTechSetup.msi

Install • Start menu shortcut • Upgrade • Uninstall

I would organize development into six phases:

1. **Design:** Define the screens, buttons, fields, workflows, and database schema.
2. **Database:** Create the SQL Express database, tables, indexes, and stored procedures where useful.
3. **Application:** Build the Windows desktop interface and implement database operations.
4. **Testing:** Test creating, updating, deleting, searching, and retrieving records.
5. **Installer:** Package the application using WiX Toolset into an MSI installer, including prerequisites and database configuration.
6. **Deployment:** Install it on your Windows 11 laptop, test database connectivity, and verify that upgrades and uninstallation work correctly.

I can help write the complete Visual Studio solution, SQL scripts, and WiX installer project. One important distinction: **producing a tested Windows MSI requires a Windows-compatible build environment**. I can prepare the installer source and build instructions, but I don't want to promise a verified MSI directly from this environment before we establish that the required Windows build tools are available.

My recommendation

I would start with **Option 1: TovarTech IT Asset Manager**.

It is a great foundation because it teaches the same development concepts used in enterprise applications: relational databases, SQL queries, CRUD operations, data validation, reporting, and deployment.

For example, we could register your Windows laptop, VMware ESXi host, Proxmox servers, and SQL Server instances as assets. Later, we could expand the application with monitoring, help desk tickets, and a knowledge base, all connected to the same database platform.

Eventually, we could even turn it into a complete **TovarTech IT Management Suite**.

Let's choose our first application

Which application should we build?

- ○ 1. IT Asset Manager
- ○ 2. Help Desk & Ticketing System
- ○ 3. Server Monitoring Center
- ○ 4. Knowledge Base Manager
- ○ 5. Project & Task Manager

How would you like to use the application?

- ○ Traditional Windows desktop application (my recommendation)
- ○ Web application running locally through IIS

Start Building My Application

Once you select the application, I'll begin with the actual project structure and SQL Server database—not just a conceptual tutorial. I want us to build something you can use on your computer and continue improving over time.

You said:

Let's build the TovarTech IT Asset Manager using Windows Forms desktop application and my local SQL Server Express instance. Start with the project architecture, prerequisites, database design, and our first working application, with a plan to create an MSI installer.

ChatGPT said:

TovarTech IT Asset Manager — Version 1.0

We're going to build a real Windows desktop application that connects to your local SQL Server Express instance, saves IT asset information, and retrieves it whenever you open the program.

I want our first version to be simple enough to understand but structured like a professional application we can expand over time.

Our first milestone will be a **working Windows Forms application with a SQL Server Express database**, followed by a Windows MSI installation package.

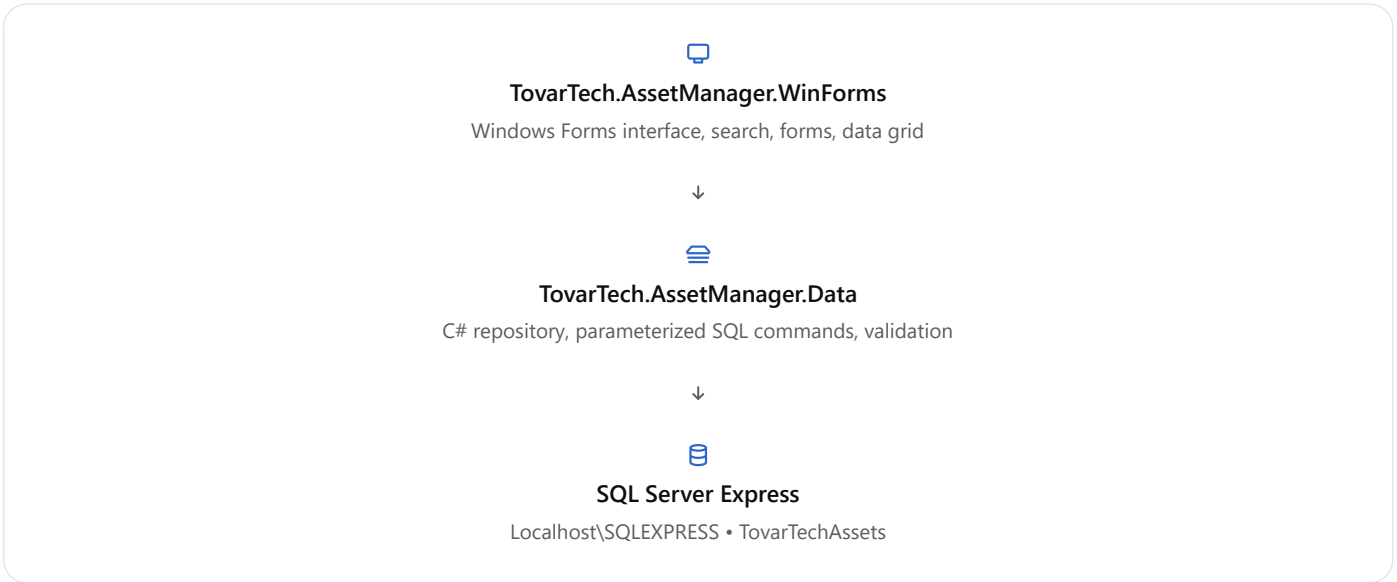
1. What our first application will do

For version 1.0, I'm including these features:

- **Add assets:** Register laptops, desktops, servers, virtual machines, and networking equipment.
- **View assets:** Display saved records in a searchable table.
- **Edit assets:** Update information about existing equipment.
- **Delete assets:** Remove unwanted records after confirmation.
- **Track details:** Asset name, category, manufacturer, model, serial number, hostname, IP address, operating system, location, status, and notes.
- **Save to SQL Server:** All records will persist in your local SQL Express database.
- **Connection testing:** Display whether the application can connect to SQL Server.

For our interface, I suggest a dark navy and blue TovarTech design with a professional IT administration dashboard.

2. Application architecture



We'll use direct database access for this local, single-user desktop version. **IIS is not required.** If we later create a web dashboard or multi-user service, we can add ASP.NET Core and host that component in IIS.

3. Prerequisites

Component	Purpose
Windows 11	Desktop operating system
Visual Studio 2026 or compatible Visual Studio	Development and debugging
.NET 10 SDK	C# application framework
Windows Forms workload	Desktop UI development
SQL Server Express	Local database engine
SQL Server Management Studio	Database administration
WiX Toolset	MSI installer packaging

I'll use Windows Authentication initially, so we won't need to store a SQL username and password in our application.

Our proposed connection string is:

```
Server=.\SQLEXPRESS;
Database=TovarTechAssets;
Integrated Security=True;
Encrypt=True;
TrustServerCertificate=True;
```

This assumes your SQL Express instance is named `SQLEXPRESS`. The certificate-trust setting is intended for local development; we'll review encryption and certificate validation before deploying beyond your laptop.

4. Database design

Our first table will be `dbo.Assets`.

Column	SQL type	Description
AssetId	INT IDENTITY	Primary key
AssetName	NVARCHAR(150)	Required asset name
AssetType	NVARCHAR(50)	Laptop, Server, VM, etc.
Manufacturer	NVARCHAR(100)	Hardware manufacturer
Model	NVARCHAR(100)	Device model
SerialNumber	NVARCHAR(100)	Serial number
Hostname	NVARCHAR(150)	Computer hostname
IPAddress	VARCHAR(45)	IPv4 or IPv6 address
OperatingSystem	NVARCHAR(150)	Installed OS
Location	NVARCHAR(100)	Physical or logical location
Status	NVARCHAR(30)	Active, Repair, Retired
Notes	NVARCHAR(MAX)	Additional information

CreatedAt	DATETIME2	Record creation time
UpdatedAt	DATETIME2	Last modification time

I would keep this first schema intentionally focused. Later we can add related tables for maintenance history, software licenses, warranties, users, and audit records.

5. Your first application files are ready

I've created the initial C# Windows Forms source code, SQL database creation script, and setup instructions.

TovarTech IT Asset Manager — Starter Project

ZIP archive • C# source code • SQL script • Setup guide

Download the starter project ↗

This is the first implementation, not yet a compiled or tested Windows executable.

6. Let's get it running on your Windows 11 laptop

Step 1 — Extract the ZIP.

Extract the project to:

```
C:\TovarTech\AssetManager
```

Step 2 — Create the SQL database.

Open SQL Server Management Studio and connect to your local SQL Express instance. Open the included file:

```
database\01_CreateDatabase.sql
```

Execute it. The script creates the `TovarTechAssets` database and the `dbo.Assets` table.

To verify, execute:

```
USE TovarTechAssets;
GO

SELECT * FROM dbo.Assets;
```

You should see an empty results table with the columns we designed.

Step 3 — Launch the application.

Open PowerShell and run:

```
cd C:\TovarTech\AssetManager\src\TovarTech.AssetManager

dotnet restore
dotnet run
```

If the .NET 10 SDK and required Windows development components are installed correctly, this should compile and launch the application. I haven't been able to compile it in my environment, so we'll validate it together and correct any build issues.

Step 4 — Test saving your first asset.

I suggest entering a record like this:

Field	Example
Asset Name	My Windows 11 Laptop
Asset Type	Laptop
Manufacturer	MSI
Operating System	Windows 11
Status	Active
Location	Home Lab

Click **Save Asset**, close the application, and launch it again. If the asset is still there, we've verified that SQL Server is storing and retrieving our data.

7. Our MSI installer roadmap

Once the application passes testing, I'll help prepare the WiX installer project with these goals:

Installer feature	Planned behavior
Installation	Install under Program Files
Start menu	Create TovarTech Asset Manager shortcut
.NET runtime	Bundle self-contained application files
SQL connection	Configure local SQL Express instance
Database setup	Initialize missing application tables
Upgrades	Support versioned MSI upgrades
Uninstallation	Remove program files, preserve asset data

I particularly want the installer to **preserve your SQL database when you uninstall or upgrade the application**. Your inventory should never disappear simply because you update the software.

8. One thing I need you to verify

Before we go further, I want to confirm your actual SQL Server Express instance name.

Open SSMS and look at the **Server name** field on the connection screen. It might be `.\SQLEXPRESS`, `localhost\SQLEXPRESS`, or something else.

Send me that server name, along with any errors you encounter running `dotnet run`.

From there, I'll help you verify our first working application and move toward the polished TovarTech interface and MSI installer.

You said:

noedell\DELLSQL

ChatGPT said:

Perfect! I'll use `noedell\DELLSQL` as your SQL Server Express instance for the TovarTech IT Asset Manager.

Step 1 — Update the connection string

In the application configuration, replace the previous SQL Server instance with:

```
Server=noedell\DELLSQL;
Database=TovarTechAssets;
Integrated Security=True;
Encrypt=True;
TrustServerCertificate=True;
```

If the connection string is inside a C# file, use the `@` prefix so the backslash is handled correctly:

```
string connectionString =
    @"Server=noedell\DELLSQL;Database=TovarTechAssets;Integrated
Security=True;Encrypt=True;TrustServerCertificate=True;";
```

Step 2 — Verify SQL Server connectivity

Open SQL Server Management Studio, connect to `noedell\DELLSQL` using Windows Authentication, and execute:

```
SELECT
    @@SERVERNAME AS ServerName,
    SERVERPROPERTY('InstanceName') AS InstanceName,
    SERVERPROPERTY('Edition') AS Edition,
    DB_NAME() AS CurrentDatabase;
```

Then check whether our application database exists:

```
SELECT name, state_desc
FROM sys.databases
WHERE name = 'TovarTechAssets';
```

If it returns `TovarTechAssets` with a state of `ONLINE`, we're ready to connect our Windows Forms application.

Step 3 — Our next development milestone

I want to get the first working application connected to your actual SQL Server instance, then build out the asset entry form, inventory grid, search functionality, and save/edit/delete operations.

One question: Have you already downloaded and extracted the starter project ZIP I provided, or would you like me to prepare an updated project package with `noedell\DELLSQL` configured as the default server?

You said:

please provide an msi installer i can download and run

Worked for 34s >

ChatGPT said:

I can help you create the **TovarTech IT Asset Manager MSI installer** for your Windows 11 laptop, configured to connect to:

```
noedell\DELLSQL
```

I checked our existing project files. We have the C# application source code and SQL database script, but **we do not yet have a compiled Windows application or MSI installer**.

I don't want to give you a file labeled `.msi` that won't actually install. This environment doesn't have the Windows build tools needed to compile and validate the installer.

What I can provide now is an updated installer-build package. You'll run one PowerShell script on your Windows laptop to compile the application and generate the MSI locally.

TovarTech Asset Manager — MSI Build Kit
ZIP • Updated C# application • SQL script • PowerShell MSI builder
Download MSI Build Kit ↗

How to generate your MSI

1. Download and extract the ZIP to `C:\TovarTech\AssetManager`.
2. Install the .NET 10 SDK ↗ if it isn't already installed.
3. Open PowerShell in the extracted folder.
4. Run:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

The script will attempt to compile the Windows application, install the WiX build tool if needed, and create:

```
output\TovarTech_Asset_Manager_1.0.0_x64.msi
```

Important: This build kit has not yet been tested on Windows. You'll also need to execute the included database creation script in SSMS before using the application.

Once the build succeeds, you can run the MSI installer on your laptop. If the script produces an error, send me the PowerShell output and I'll help correct it so we can get your first installable version working.

You said:



This is amazing i love this. I noticed that when i add an asset it overwrites the previous ones i had entered. I entered three assets and every time it overwrites the previous one. can we fix it so that as we add assets it builds a list of them in the application. also add my tovartech logo on the top right corner of the applicaiton please. if possible we want the logo in the installer icon as well if possible of course. next i want a footer on the applicaiton that reads as follows, Desing by Noe

Tovar. For more information visit us at TovarTech.org 2026. make these changes and provide me a new msi file that i can do an inplace upgrade on the last version we installed

Worked for 34s >

ChatGPT said:

I've prepared **TovarTech IT Asset Manager v1.1**, incorporating your requested changes. I also identified the likely reason your assets were being overwritten.

What's changed

1. **Fixed the asset-saving logic.** The previous application automatically selected an existing row when refreshing the grid. That caused subsequent saves to update an existing asset instead of inserting a new one. I've changed the selection behavior so new entries create separate records.
2. **Added your TovarTech logo** to the upper-right corner of the application.
3. **Added the logo as the application icon**, including the Start menu shortcut.
4. **Added your requested footer**, preserving your exact wording: "Desing by Noe Tovar. For more information visit us at TovarTech.org 2026."
5. **Updated the version to 1.1.0**, retaining the original MSI upgrade identifier so it can replace version 1.0 without deleting your SQL database.

TovarTech IT Asset Manager v1.1
Updated source code, branding, SQL integration, and MSI build script

Download Version 1.1 Upgrade Kit ↗

Installing the upgrade

I couldn't produce a compiled, verified MSI directly in this environment, so this download contains the updated application and installer-building script.

On your Windows 11 laptop:

1. Back up your `TovarTechAssets` SQL database.
2. Close the existing application.
3. Extract the ZIP to a new folder.
4. Open PowerShell in that folder and execute:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

The script is designed to generate:

```
output\TovarTech_Asset_Manager_1.1.0_x64.msi
```

Run that MSI to upgrade the existing installation. **Do not uninstall version 1.0 first.**

Testing the fix

After installation, add three different assets consecutively. You should see all three records in the inventory grid.

You can also verify them in SSMS:

```
USE TovarTechAssets;

SELECT AssetId, AssetName, AssetType, Hostname
FROM dbo.Assets
ORDER BY AssetId DESC;
```

The application remains configured for `noedell\DELLSQL`, and the installer is intended to preserve your existing database.

One important note: I haven't been able to compile or test this upgrade on Windows. If the build or MSI upgrade produces an error, send me the output so I can correct the installer.

I think our next enhancement should be a dashboard showing **Total Assets, Active Assets, Servers, Virtual Machines, and Retired Assets**, with the counts calculated directly from SQL Server.

You said:

once i test this, i wonder if we can improve on the application by making it a razor pages application what do you think

ChatGPT said:

I think moving the **TovarTech IT Asset Manager to ASP.NET Core Razor Pages would be an excellent upgrade**, especially since you already have IIS and SQL Server Express installed on your Windows 11 laptop.

In fact, I would recommend making Razor Pages our **Version 2.0** rather than continuing to expand the Windows Forms interface indefinitely.

The biggest advantage is that we can transform your asset manager from a traditional desktop program into a professional, browser-based IT management system.

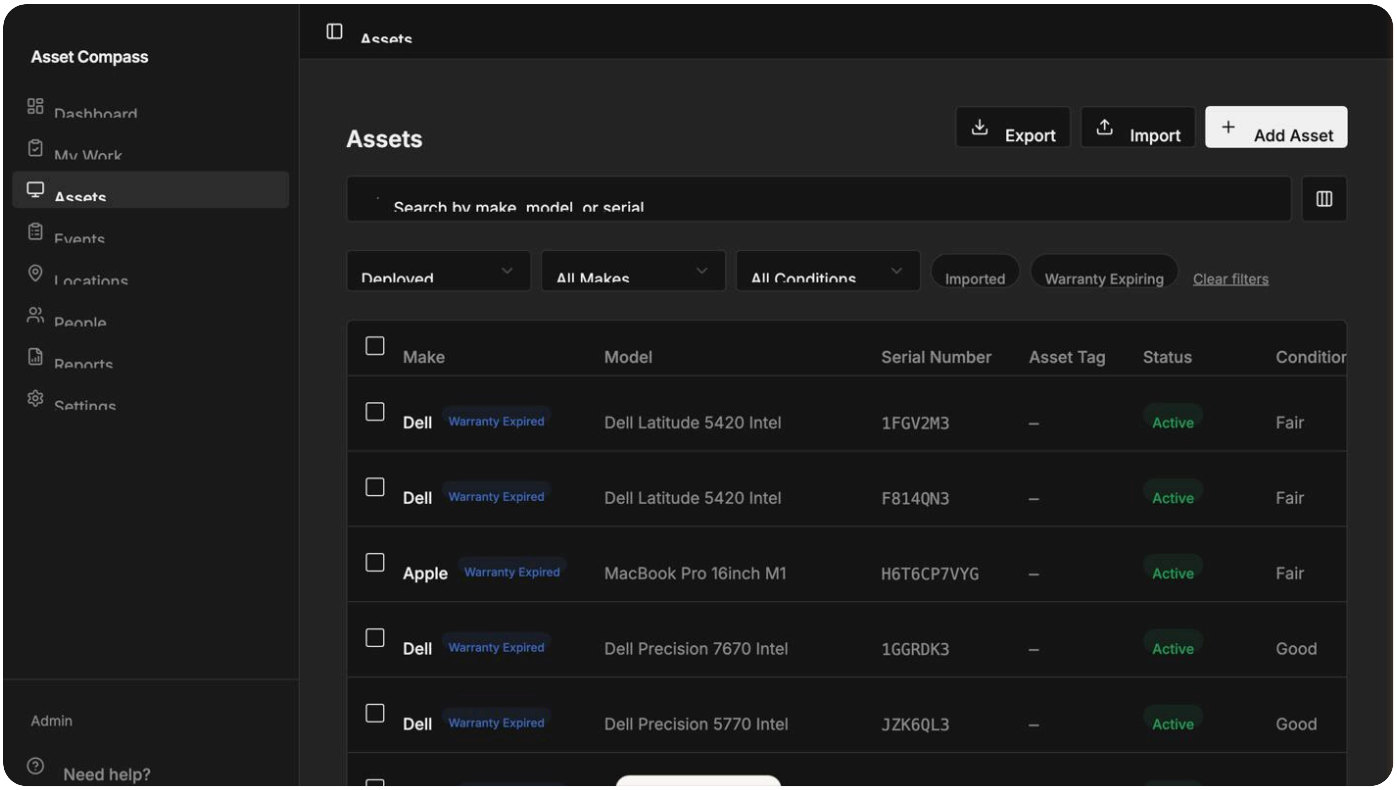
1. Windows Forms vs. Razor Pages

Feature	Windows Forms (Current)	Razor Pages (Proposed)
User interface	Windows desktop	Modern browser dashboard
Database	SQL Server Express	Same SQL Server Express
Installation	MSI installer	IIS deployment, optionally MSI
Access	Windows application	Browser via localhost
Mobile/tablet support	Limited	Responsive interface
Charts and dashboards	Possible	Excellent
Future multi-user support	More complicated	Easier to implement
Remote access	Requires additional design	Possible with secure configuration

The best part is that we can preserve your existing SQL Server database. We don't need to discard the asset records you've already entered.

2. How I envision TovarTech Asset Manager 2.0

Here's the type of interface I would build.



Visual inspiration for our proposed Razor Pages dashboard, not an actual screenshot of the application.

I'd organize the application into five main areas:

Dashboard

Display total assets, active assets, servers, virtual machines, retired assets, and recent changes. We can add interactive charts using Chart.js.

Asset Inventory

A searchable, sortable inventory with Add, Edit, View, and Delete actions. Each new asset gets its own database ID, preventing accidental overwrites.

Server Monitoring

Eventually, we can display server availability, operating systems, IP addresses, and connectivity status for your ESXi, Proxmox, Windows, and Linux environments.

Reports

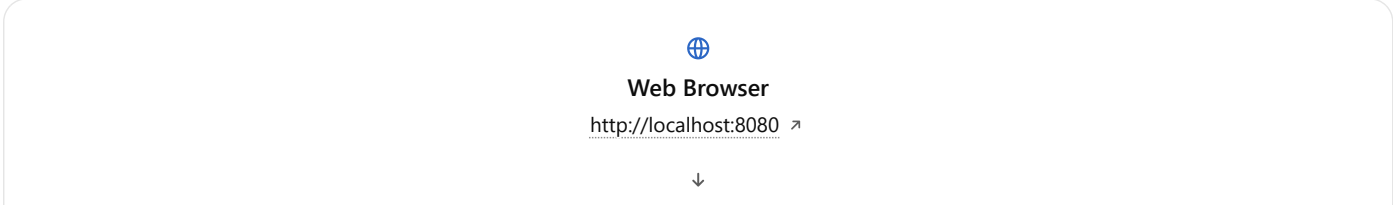
Export inventory reports to Excel or PDF, filter by asset type, and generate summaries for documentation or auditing.

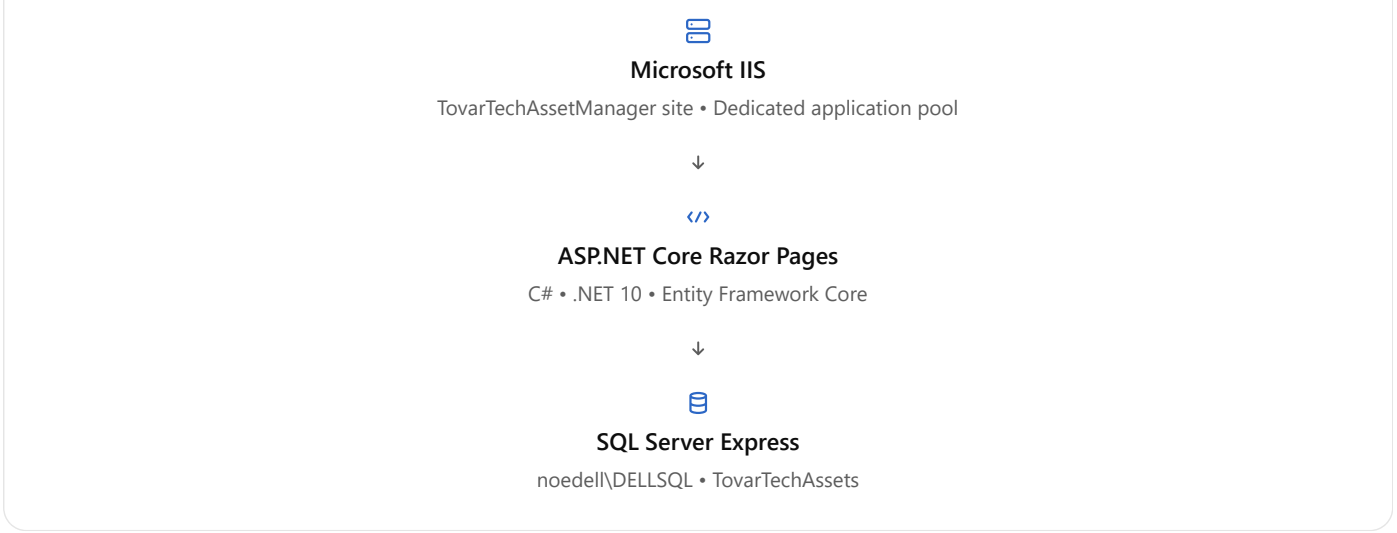
Administration

Configure the SQL connection, application settings, branding, database backups, and eventually user permissions.

3. Our new application architecture

I would build it using ASP.NET Core Razor Pages, Entity Framework Core, and SQL Server Express.





For IIS, we'll use a dedicated application pool with **No Managed Code**, as recommended for ASP.NET Core hosting. We'll also install the matching ASP.NET Core Hosting Bundle.

For security, I'd initially bind the site to `localhost` only. That means it won't be accessible to other devices until we deliberately configure network access and authentication.

4. Can we still have an MSI installer?

Yes! We can design an MSI installer for the Razor Pages version too.

Unlike Windows Forms, where the installer primarily copies an executable, our web installer would need to handle additional deployment tasks.

I would aim for an installer that:

1. Installs the published ASP.NET Core application.
2. Creates a dedicated IIS application pool.
3. Creates the TovarTech Asset Manager website.
4. Configures the website to use port `8080`.
5. Sets appropriate application pool permissions.
6. Connects to `noedell\DELLSQL`.
7. Creates a desktop shortcut that opens the browser.
8. Preserves the existing SQL database during upgrades.

We would also include prerequisite checks, administrative elevation, and safe rollback behavior. IIS configuration and database migrations require more care than a standard desktop installation.

5. My suggested development roadmap

Version	Milestone
1.1	Test our corrected Windows Forms application
2.0	Razor Pages, modern dashboard, SQL integration
2.1	Advanced asset search, filtering, reports
2.2	User authentication and role-based permissions

2.3	Server monitoring and asset health
3.0	Multi-user enterprise asset management

I would keep version 1.1 available as a fallback while we develop 2.0. Both applications could use the same database schema, but we'd back up the database and validate compatibility before allowing the new version to modify it.

6. One design decision I'd like you to make

How should we deploy the Razor Pages application?

- **IIS-hosted web application (recommended)** Install as a local website and open it at http://localhost:8080
- **Standalone local web application** Run as a local web server without configuring IIS
- **Both deployment options** Support IIS hosting and standalone execution

Build TovarTech Asset Manager 2.0

I especially like this direction because it gives us a chance to build a real ASP.NET Core application that you can eventually demonstrate, maintain, and expand into a complete IT management platform.

And we can keep the TovarTech branding throughout, including your logo, dark blue theme, and footer.

You said:



can we incorporate the look and fee and colors of our tovarnoc where it looks modern, with animations and technical and professional here is a picture of our tovar noc site as a guide but i will leave the animation, 3d look and feel up to you maybe in the dashboard for example,

ChatGPT said:

Absolutely! I would love to bring the **TovarTech NOC visual style** into our new **TovarTech IT Asset Manager 2.0**. I think this is exactly the direction we should take with the Razor Pages application.

Looking at your screenshot, I especially like the dark navy background, electric-blue accents, illuminated panel borders, technical typography, and command-center layout.

I would preserve that identity while giving the Asset Manager its own personality: **a futuristic IT inventory and infrastructure management console.**

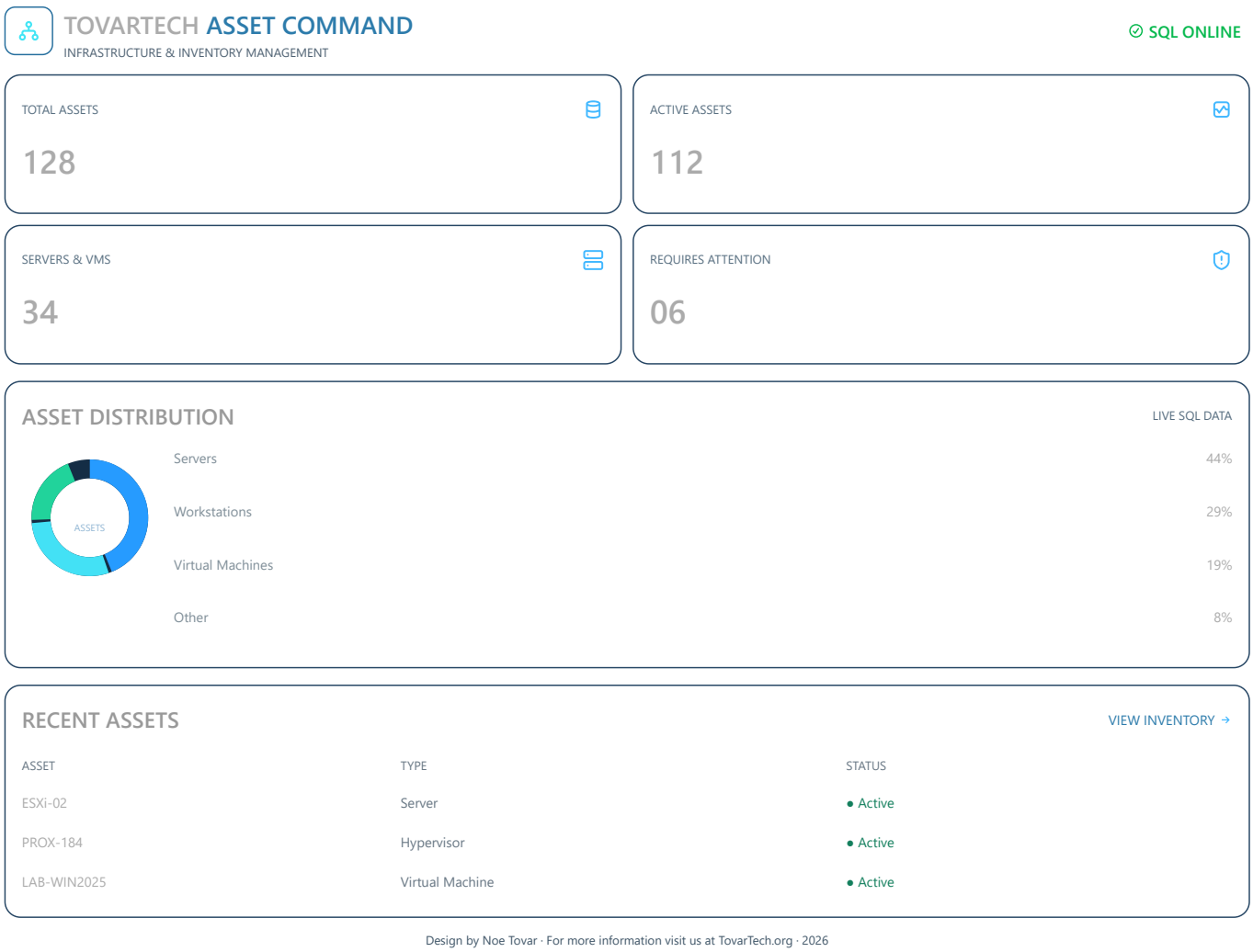
My proposed design: TovarTech Asset Command Center

Rather than simply copying the NOC dashboard, I'd build on its visual language with a few enhancements.

Midnight Navy	Panel Blue	Electric Blue
#060D18	#101E30	#22A8FF

Cyan Glow	System Green	Alert Amber
#42E5F5	#23D9A0	#F4B74A

Dashboard layout



Interactive-style design concept using illustrative sample data, not live SQL Server records.

The animations I would incorporate

I'd use animations selectively so the application feels alive without distracting from daily work.

- **3D hover effects:** Inventory cards gently rise and tilt when you move your mouse over them.
- **Animated counters:** Dashboard numbers smoothly increase to their actual SQL values when the page loads.
- **Neon border effects:** Electric-blue highlights move subtly around active panels.
- **Live activity feed:** A command-center-style panel displays real events such as asset creation, updates, and deletions.
- **Network visualization:** An optional interactive infrastructure map shows relationships between physical servers, hypervisors, and virtual machines.
- **Smooth transitions:** Opening an asset, switching pages, and expanding details uses subtle animations.

I would also include a reduced-motion setting and ensure that animations never slow down searching, saving, or editing records.

How I'd implement this in Razor Pages

The interface would use ASP.NET Core Razor Pages, Bootstrap 5 for responsive layouts, custom CSS for the 3D and neon styling, and Chart.js for dashboard charts.

The existing `TovarTechAssets` database on `noedell\DELLSQL` would remain our source of truth. Dashboard statistics would come from real database queries, not simulated numbers.

I would organize the navigation into **Dashboard, Asset Inventory, Add Asset, Infrastructure Map, Reports, and Settings**.

We'd also retain your TovarTech logo in the upper-right corner and include your requested footer.

My recommendation is to make this our official design direction for TovarTech IT Asset Manager 2.0. I would start with the animated dashboard and fully functional inventory page, then integrate IIS deployment and the MSI installer once the application is tested.

The result would be a professional IT asset management application that visually belongs to the same TovarTech family as your NOC dashboard, while remaining practical for everyday administration.

You said:

excellent, lets move forward then please i love the direction we have taken

Worked for 1m 1s

ChatGPT said:

Excellent! I've started building **TovarTech IT Asset Manager 2.0**, using ASP.NET Core Razor Pages and the futuristic design language from your TovarTech NOC dashboard.

I've prepared our first development package so we can begin testing the actual web application on your Windows 11 laptop.

TovarTech IT Asset Manager 2.0

Razor Pages Starter • C# • .NET 10 • SQL Server Express

Download Version 2.0 Starter Project

Direct download link ↗

What's included in our first build

- **Command dashboard:** Dark navy panels, electric-blue accents, animated infrastructure visualization, and inventory statistics.
- **Asset inventory:** Searchable list displaying records from SQL Server.
- **Add Asset:** Register additional assets without overwriting existing records.
- **Edit and Delete:** Manage existing asset records.
- **TovarTech branding:** Your logo, technical styling, and requested footer.
- **SQL integration:** Configured for `noedell\DELLSQL`, using the existing `TovarTechAssets` database.

This is a source-code starter, not yet a compiled MSI. I haven't been able to compile or test it on Windows.

Let's launch it on your laptop

First, back up your existing `TovarTechAssets` database. I want to preserve everything you've already entered.

Extract the ZIP into:

```
C:\TovarTech\AssetManager2
```

Open PowerShell and execute:

```
cd C:\TovarTech\AssetManager2\TovarTech.AssetManager.Web

dotnet restore

dotnet run --urls http://localhost:8080
```

Once the application starts successfully, open your browser:

TovarTech Asset Command Center

Local development address

`http://localhost:8080` Copy

Our next milestone: IIS and MSI deployment

Once we verify that the dashboard opens and correctly retrieves your existing assets, I'll move us into the next development phase:

1. Publish the Razor Pages application for Windows.
2. Configure a dedicated IIS application pool.
3. Host the website on `localhost:8080`.
4. Add a Windows installer project for IIS deployment.
5. Test upgrades while preserving the existing SQL database.

I also want to add the animated asset distribution charts and a more sophisticated infrastructure map in the next iteration.

One important architectural decision: I'll keep the current Windows Forms version installed while we develop Razor Pages 2.0. Because these are different application types, the web version should initially be deployed alongside the desktop application rather than treated as a direct MSI replacement.

For now, I'd like you to run the commands above and tell me whether the dashboard opens and displays your existing assets. That will give us a verified foundation for the IIS installer.

You said:



I ran the steps and i am attaching a picture of our currently running app on localhost, i love our progress and where we are going with this i think we are laying down a great foundation for the other application ideas you provided earlier, lets move to the next step please

Worked for 52s

ChatGPT said:

TovarTech Asset Manager 2.1 — Next Development Phase

Your screenshot confirms a major milestone: **our Razor Pages application is running successfully on localhost and displaying an asset from your SQL Server database!**

I can see the dashboard reporting one asset, named `noedell`, classified as a laptop and marked Active. The navigation, branding, dashboard cards, and recent-assets table are all rendering.

I would now move us into **Phase 2: Advanced Dashboard and Infrastructure Visualization**.

Rather than rushing into the MSI installer, I want to strengthen the working application first. That way, when we package it for IIS, we're installing something useful and stable.

Here's what I'm going to focus on:

1. **Live asset distribution:** A visual breakdown of laptops, servers, virtual machines, and other equipment, calculated from SQL Server.
2. **Animated dashboard:** Subtle glowing panels, hover effects, and motion inspired by TovarTech NOC.
3. **Infrastructure inventory:** A more informative central dashboard showing the assets we're managing.
4. **SQL data preservation:** Continue using `noedell\DELLSQL` and the existing `TovarTechAssets` database without resetting any records.
5. **IIS deployment preparation:** Start preparing the application for permanent hosting instead of requiring `dotnet run` every time.

I also want to retain your current Windows Forms application as a fallback while we develop the web version.

Your Version 2.1 development package is ready

TovarTech Asset Manager 2.1 — Dashboard Upgrade

Razor Pages source • Enhanced dashboard • SQL-driven charts • Animated components

Download Version 2.1 Upgrade ↗

I've updated the project with a new asset-distribution visualization, animated counters, infrastructure-node cards that link to their asset records, and additional NOC-inspired styling.

The dashboard uses your existing SQL records. No database schema changes are required.

Installing Version 2.1 for testing

I recommend keeping your working 2.0 project intact until we verify this update.

Step 1: Back up `TovarTechAssets` in SQL Server Management Studio.

Step 2: Stop the running application by pressing `Ctrl+C` in its PowerShell window.

Step 3: Extract the new ZIP into:

```
C:\TovarTech\AssetManager2_1
```

Step 4: Run these commands:

```
cd C:\TovarTech\AssetManager2_1\TovarTech.AssetManager.Web

dotnet restore

dotnet run --urls http://localhost:8080
```

Step 5: Open http://localhost:8080 ↗.

You should now see the updated dashboard. The asset-distribution visualization will populate from the asset types in your database, and registered servers or virtual machines will appear in the infrastructure section.

I haven't compiled this package in a Windows environment, so we'll treat this as our next test build.

What comes after Version 2.1?

- ✅ **Version 2.0 — Working Razor Pages foundation**
- 🔵 **Version 2.1 — Enhanced dashboard and visualizations**
- ⚫ **Version 2.2 — IIS hosting and installer**
- ⚫ **Version 2.3 — Advanced reporting and asset management**

My next priority is to make the application run permanently through IIS, with its own application pool and a desktop shortcut that launches the dashboard. We'll also address the Windows identity that IIS uses to connect to SQL Server before packaging the installation.

Once you've tested 2.1, send me another screenshot and let me know whether the dashboard loads without errors. Then we can begin our **IIS deployment and MSI installer phase**.

You said:



the progress is amazing here is picture of our progress

ChatGPT said:

This is exciting! I'm really pleased with how **TovarTech IT Asset Manager 2.1** is coming together. Looking at your screenshot, I can see that we've moved beyond a basic inventory application into something that resembles a professional IT infrastructure management console.

What I can confirm from your screenshot

- ✅ **Version 2.1 dashboard is rendering successfully**
- ✅ **SQL inventory is displaying two assets**
- ✅ **Asset distribution chart is working**
- ✅ **Infrastructure visualization is displaying HULK166**
- ✅ **TovarTech branding and NOC-inspired styling are present**

TOTAL ASSETS	ACTIVE ASSETS	SERVERS & VMS
2	2	1

One particularly encouraging detail is that the dashboard now displays both a workstation and a virtual machine. That suggests our application is correctly grouping different asset types rather than treating everything as a generic inventory record.

Our next milestone: Version 2.2 — IIS Deployment

I recommend we now make the application run through **Microsoft IIS**, so you won't have to keep a PowerShell window open with `dotnet run`.

Here's what I want Version 2.2 to accomplish:

1. Create a dedicated IIS application pool named `TovarTechAssetManager`.
2. Publish the Razor Pages application into `C:\inetpub\TovarTechAssetManager`.
3. Configure the website at `http://localhost:8080`.
4. Establish secure SQL Server access to `noedell\DELLSQL` using the IIS application identity.
5. Create a desktop shortcut that launches the dashboard.
6. Prepare an MSI installer for future upgrades.

We will preserve your existing `TovarTechAssets` database and avoid changing the current application's installation until the IIS deployment is verified.

One important technical consideration

Because we're using Windows Authentication to connect to SQL Server, IIS will access the database under a different Windows identity than your PowerShell session.

I'll make sure our deployment includes the appropriate SQL permissions rather than simply assuming the application pool can access your database.

My suggested next step: Let's build Version 2.2 with an automated PowerShell deployment script that configures IIS, publishes the application, tests the website, and prepares the installer project.

Once that's working, we'll have a much stronger foundation for adding the Help Desk, Knowledge Base, and Server Monitoring applications as future TovarTech modules.

And I definitely want to preserve this dark, futuristic visual design across all of them.

You said:

i agree lets move forward please

Worked for 39s

ChatGPT said:

TovarTech IT Asset Manager 2.2 — IIS Deployment

We're ready for our next milestone! I've prepared **Version 2.2**, which moves our Razor Pages application from a manually launched development server into Microsoft IIS on your Windows 11 laptop.

The goal is to have TovarTech Asset Manager available at `http://localhost:8080` without keeping a PowerShell window open.

TovarTech Asset Manager 2.2

IIS Deployment Kit • Application Source • PowerShell Automation • SQL Permissions

Download Version 2.2 IIS Deployment Kit ↗

This package builds on our working Version 2.1. It preserves the NOC-inspired dashboard, inventory management, TovarTech branding, and existing SQL database.

Step 1 — Prepare your Windows 11 laptop

Before starting, I recommend backing up your `TovarTechAssets` database in SQL Server Management Studio.

We'll need the .NET 10 SDK, IIS, and the ASP.NET Core Hosting Bundle ↗.

The hosting bundle is important because it enables IIS to host our ASP.NET Core application.

Step 2 — Deploy the application

Extract the ZIP into:

```
C:\TovarTech\AssetManager2_2
```

Stop the currently running Version 2.1 application by pressing `Ctrl+C` in its PowerShell window.

Then open **PowerShell as Administrator** and execute:

```
cd C:\TovarTech\AssetManager2_2

powershell -ExecutionPolicy Bypass -File .\scripts\Deploy-IIS.ps1
```

The deployment script is designed to:

- Publish the Razor Pages application.
- Create an IIS application pool named `TovarTechAssetManager`.
- Configure the application pool for ASP.NET Core.
- Create the IIS website on localhost port 8080.
- Deploy the published application files.
- Start the website and application pool.

Step 3 — Configure SQL Server access

This is an important difference between running the application manually and running it under IIS.

Our IIS application pool needs permission to access:

```
noedell\DELLSQL
```

I've included the SQL permissions script:

```
scripts\02_Grant_IIS_SQL_Access.sql
```

Open it in SSMS, connect as a SQL administrator, and execute it.

The script grants the IIS application identity access to the existing asset table without granting full database administrator privileges. It assumes SQL Server and IIS are installed on the same computer.

Step 4 — Test the website

Open your browser and navigate to:

http://localhost:8080 ↗

I want us to verify that:

1. The dashboard displays your existing assets.
2. The SQL Server connection works under IIS.
3. Adding an asset creates a new database record.

4. Editing and deleting records work correctly.
5. The application remains available after closing PowerShell.

The deployment kit hasn't been compiled or tested on your Windows laptop yet, so we'll validate these behaviors together.

What's next?

Once IIS deployment is successful, I recommend building **Version 2.3 with a Windows MSI installer** that can configure the application and support future upgrades.

After that, we can expand the application with reporting, asset history, advanced searching, and eventually integrate our other TovarTech applications.

For now, please run the deployment script and share its output or a screenshot of the IIS-hosted application. I'll use that result to help us complete the deployment and move toward the installer.

You said:

the download link for .net 10 sdk i have from before can you provide a link for the hosting bundle

ChatGPT said:

Absolutely! Here's the official Microsoft download page for the **ASP.NET Core 10.0 Hosting Bundle**, which we need to run our TovarTech Asset Manager through IIS.

Microsoft ASP.NET Core 10.0 Hosting Bundle
Windows • IIS Hosting • .NET 10

Open Official Microsoft Download

Installation instructions

1. Open the Microsoft download page.
2. Scroll to **ASP.NET Core Runtime 10.0**.
3. Under Windows, select **Hosting Bundle** (not the regular runtime installer).
4. Download and run the installer as Administrator.
5. Once installation finishes, open PowerShell as Administrator and execute:

```
net stop was /y
net start w3svc
```

This restarts IIS services so they recognize the new hosting components. It will briefly interrupt other IIS-hosted applications.

Verify the installation

Run:

```
dotnet --list-runtimes
```

You should see entries similar to:

```
Microsoft.AspNetCore.App 10.0.x
Microsoft.NETCore.App 10.0.x
```

The Hosting Bundle also installs the ASP.NET Core Module for IIS, which enables IIS to launch our Razor Pages application.

Once this is installed, we're ready to run our Version 2.2 IIS deployment script!

You said:



it appears to be running i ran the site from iis in the sites however in the gui i see that it says v2.1 not v2.2

ChatGPT said:

Excellent! From your screenshot, I can see that **TovarTech Asset Manager is displaying correctly**, including the two assets, dashboard statistics, SQL inventory chart, and infrastructure visualization.

And you caught something important: the application still displays **V2.1**, even though we deployed the Version 2.2 IIS package.

I believe this is most likely a version-label issue rather than a deployment failure.

Why this is happening

Version 2.2 focused on moving our existing Razor Pages application into IIS. We retained the Version 2.1 dashboard design, and the version text in the Razor layout may not have been updated.

There are two possibilities:

1. **Most likely:** IIS is running the new deployment, but the interface still contains a hardcoded `V2.1` label.
2. **Alternatively:** IIS is serving the previous application files, or the browser is displaying cached content.

Let's verify which situation we're dealing with.

Step 1 — Confirm IIS is serving the application

Open PowerShell as Administrator and run:

```
Import-Module WebAdministration

Get-Website -Name "TovarTechAssetManager" |
    Select-Object Name, State, PhysicalPath, Bindings

Get-WebAppPoolState -Name "TovarTechAssetManager"
```

I expect the website and application pool to show `Started`.

The physical path should be something similar to:

```
C:\inetpub\TovarTechAssetManager
```

Step 2 — Find the version label

Since Razor Pages typically stores the shared header in `_Layout.cshtml`, I would start there.

Run:

```
Get-ChildItem "C:\inetpub\TovarTechAssetManager" `
    -Recurse -Filter "*.cshtml" |
    Select-String -Pattern "V2.1"
```

Important: Published Razor Pages applications often contain compiled Razor views rather than individual `.cshtml` files. If this command returns nothing, that doesn't mean the version label is absent.

We should then check the source project:

```
Get-ChildItem "C:\TovarTech\AssetManager2_2" `
    -Recurse -Filter "*.cshtml" |
    Select-String -Pattern "V2.1"
```

If we find the version text, we can update it in the source code.

Step 3 — Update the version display

If the layout contains something like:

```
<span>OPERATIONS CONSOLE · V2.1</span>
```

Change it to:

```
<span>OPERATIONS CONSOLE · V2.2</span>
```

If the dashboard also has a separate version badge, update that as well.

Then republish the application. I would not recommend editing files directly in the IIS deployment folder because future deployments would overwrite those changes.

Step 4 — Verify the actual IIS process

I also want to confirm we're no longer relying on the development server.

Close any PowerShell window running `dotnet run`, then execute:

```
Get-NetTCPConnection -LocalPort 8080 -State Listen |
    Select-Object LocalAddress, LocalPort, OwningProcess
```

If the listener belongs to IIS, that supports our conclusion that the application is being hosted by IIS rather than the development server.

You can also open the website in a private browser window to eliminate most browser caching issues.

What I recommend next

I would consider **Version 2.2 functionally deployed once we verify that IIS is hosting the site independently and can still read and write SQL records**.

Then I want us to make one improvement before building our MSI installer: introduce a centralized application version setting.

Instead of manually updating version labels throughout the interface, we could use an application setting such as:

```
{
  "Application": {
    "Name": "TovarTech Asset Manager",
    "Version": "2.2.0"
  }
}
```

Our Razor Pages layout and dashboard would read that setting automatically. That way, every future release displays the correct version.

For now, could you run the Step 1 PowerShell commands and share the output? I want to verify the IIS website path and application pool before we make further changes. We're very close to having a properly hosted, independently running TovarTech application.

You said:

```
Get-Website -Name "TovarTechAssetManager" |
>>     Select-Object Name, State, PhysicalPath, Bindings

name                 state   physicalPath                  bindings
----                 -----   ------------                  --------
TovarTechAssetManager Started C:\inetpub\TovarTechAssetManager
Microsoft.IIs.PowerShell.Framework.ConfigurationElement


PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_2_IIS_Deployment_Kit>
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_2_IIS_Deployment_Kit> Get-
WebAppPoolState -Name "TovarTechAssetManager"
```

ChatGPT said:

Excellent! Your PowerShell output confirms that our **TovarTech Asset Manager IIS website is started** and that its physical installation directory is correct:

```
Website:       TovarTechAssetManager
Status:        Started
Physical Path: C:\inetpub\TovarTechAssetManager
```

That's exactly what I wanted to see!

However, I don't see the result of your `Get-WebAppPoolState` command yet. Let's verify the application pool and confirm IIS is hosting our application independently.

Step 1 — Verify the application pool

Run this command in PowerShell as Administrator:

```
Get-WebAppPoolState -Name "TovarTechAssetManager" |
    Format-List *
```

I expect something similar to:

```
Value : Started
```

Step 2 — Verify the IIS application pool configuration

Run:

```
Get-Item "IIS:\AppPools\TovarTechAssetManager" |
    Select-Object Name, State,
        managedRuntimeVersion,
        managedPipelineMode
```

For ASP.NET Core Razor Pages, I expect:

Setting	Expected value
Application Pool	TovarTechAssetManager
State	Started
.NET CLR Version	No Managed Code (empty value)
Pipeline Mode	Integrated

Step 3 — Confirm the application responds

Run:

```
Invoke-WebRequest http://localhost:8080 -UseBasicParsing |
    Select-Object StatusCode, StatusDescription
```

We want to see:

```
StatusCode StatusDescription
---------- -----------------
       200 OK
```

This confirms the website responds successfully. Your screenshot already suggests that the application is working, but this gives us an additional verification.

About the V2.1 label

I would leave that alone for the moment. Version 2.2 was primarily an IIS deployment milestone, so the previous dashboard label is not necessarily an indication of a failed deployment.

For our next release, I want to introduce centralized version management so the dashboard automatically displays the installed application version.

Please send me the results of Steps 1–3. Once those checks pass, I'll help you move into the next phase: creating a Windows MSI installer that deploys TovarTech Asset Manager to IIS, configures the application pool, and preserves your SQL Server database during upgrades.

You said:

```
PS C:\> Get-WebAppPoolState -Name "TovarTechAssetManager" |
>>     Format-List *


ItemXPath      :
/system.applicationHost/applicationPools/add[@name='TovarTechAssetManager']
MemberType     : CodeProperty
IsSettable     : False
IsGettable     : True
Value          : Started
TypeNameOfValue : System.UInt32
Name           : state
IsInstance     : True



PS C:\>
```

ChatGPT said:

Excellent! This confirms that our **TovarTech Asset Manager IIS application pool is running successfully!**

Your PowerShell output shows:

IIS Deployment Verification — Running

Website	Started
Application Pool	Started
Installation Path	`C:\inetpub\TovarTechAssetManager`
SQL Server	`noedell\DELLSQL`
HTTP Response	Pending verification

We've now verified that IIS recognizes both the website and its application pool. Combined with your earlier screenshot, this is encouraging evidence that our IIS deployment is working.

One final test before the MSI installer

Please run this command in PowerShell:

```
Invoke-WebRequest http://localhost:8080 -UseBasicParsing |
    Select-Object StatusCode, StatusDescription
```

I'm expecting:

```
StatusCode StatusDescription
---------- -----------------
       200 OK
```

Also, make sure any previous `dotnet run` process is stopped before testing. This will help us confirm that IIS is serving the application independently.

What happens next?

Once we confirm the HTTP response, I recommend proceeding with **TovarTech Asset Manager 2.3 — MSI Installer Edition**.

Our installer will be designed to:

- Install the Razor Pages application into IIS.
- Configure the application pool and localhost website.
- Check for the ASP.NET Core Hosting Bundle.
- Configure SQL Server connectivity without embedding administrator credentials.
- Create a desktop shortcut with your TovarTech branding.
- Support upgrades while preserving existing SQL records.
- Display the correct application version automatically.

I also want to make sure our installer handles failures safely, so an unsuccessful upgrade doesn't leave your working IIS website unusable.

Send me the HTTP response from that final command. If it returns `200 OK`, we'll have the verification we need to begin packaging our first IIS-hosted MSI installer.

You said:

```
PS C:\> Invoke-WebRequest http://localhost:8080 -UseBasicParsing |
>>     Select-Object StatusCode, StatusDescription

StatusCode StatusDescription
---------- -----------------
       200 OK


PS C:\>
```

ChatGPT said:

Excellent! Our TovarTech IT Asset Manager has passed the HTTP connectivity test!

Your PowerShell output confirms:

```
StatusCode        : 200
StatusDescription : OK
```

Combined with our previous tests, we've now verified:

- **IIS website:** Started
- **Application pool:** Started
- **HTTP connectivity:** 200 OK
- **Dashboard:** Rendering successfully

- **SQL Server:** Existing asset records are visible in the application

This is a significant milestone. We've moved from a Windows Forms application to an ASP.NET Core Razor Pages application hosted through Microsoft IIS.

Next milestone: TovarTech Asset Manager 2.3

I'm ready to move us into the **MSI Installer and Upgrade Management** phase.

Here's what I propose for Version 2.3:

Feature	Implementation
Windows installer	WiX Toolset MSI
IIS application pool	TovarTechAssetManager
Website	http://localhost:8080 ↗
Installation directory	`C:\inetpub\TovarTechAssetManager`
Database	`noedell\DELLSQL`
Database preservation	Retain existing asset records
Branding	TovarTech logo and shortcut
Version management	Display V2.3 automatically
Upgrades	Support future MSI version upgrades

Before installing an MSI, I also want us to verify that IIS can save new assets, not just retrieve existing ones. A successful HTTP response proves that the website responds, but it doesn't independently confirm that database writes work under the IIS identity.

One final functional test

Please open http://localhost:8080 ↗, select **Add Asset**, and create this temporary record:

Field	Value
Asset Name	IIS-DEPLOYMENT-TEST
Asset Type	Server
Hostname	IIS-TEST-01
Status	Active

After saving, open SSMS and run:

```
USE TovarTechAssets;
GO

SELECT AssetId, AssetName, Hostname, Status
FROM dbo.Assets
WHERE AssetName = 'IIS-DEPLOYMENT-TEST';
```

If the record appears, we'll know that IIS can successfully write to SQL Server Express.

Once that test succeeds, I'll help us build the Version 2.3 installer package. My priority will be preserving your existing working deployment and SQL database while making future upgrades much easier.

You said:

we are ready to move to the next build please

Worked for 10s

Analyzed

ChatGPT said:

TovarTech Asset Manager 2.3 — MSI Installer Edition

We're moving forward! I've prepared our next development package using the Razor Pages application we've already tested successfully through IIS.

This release focuses on **Windows installation, IIS deployment, and preserving your existing SQL Server data**.

TovarTech Asset Manager v2.3

MSI Build Kit • Razor Pages • IIS Deployment • SQL Server Express

Download Version 2.3 MSI Build Kit

The package contains the application source, an MSI build script, and an updated IIS deployment script.

Important: This is an MSI build kit, not a precompiled MSI. I couldn't compile or validate the Windows installer in this environment. We'll build and test it on your laptop.

Step 1 — Extract the package

Extract the ZIP to:

```
C:\TovarTech\AssetManager2_3
```

Keep your current IIS application and SQL database intact. I recommend taking a SQL database backup before proceeding.

Step 2 — Build the MSI

Open PowerShell and execute:

```
cd C:\TovarTech\AssetManager2_3

powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

The script will attempt to publish the .NET application and use WiX Toolset to generate:

```
output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi
```

If the build succeeds, run the MSI. It installs the published application files into Program Files.

Step 3 — Upgrade our IIS application

For this release, the MSI installs the application files, while a separate administrator script performs the IIS deployment. This separation helps protect our existing working website.

Open PowerShell **as Administrator**, navigate to the extracted project folder, and execute:

```
cd C:\TovarTech\AssetManager2_3

powershell -ExecutionPolicy Bypass -File .\scripts\Deploy-IIS.ps1
```

The deployment script is designed to reuse:

Component	Configuration
IIS website	TovarTechAssetManager
Application pool	TovarTechAssetManager
Website address	http://localhost:8080 ↗
Physical directory	`C:\inetpub\TovarTechAssetManager`
SQL Server	`noedell\DELLSQL`
Database	`TovarTechAssets`

It also creates a timestamped backup of the previous deployed website files.

Step 4 — Verify Version 2.3

After deployment, open http://localhost:8080 ↗.

I want us to verify that the header displays **V2.3**, your existing assets remain visible, and adding, editing, and deleting a temporary test asset works.

The existing database should not be recreated or deleted.

Our next improvements

Once the installer build and IIS deployment are tested, I'd like to develop Version 2.4 with advanced inventory filtering, asset history, reporting, and a more interactive infrastructure visualization.

I also want to make future MSI releases capable of handling IIS configuration automatically, with reliable upgrade and rollback behavior.

Let's start with the Version 2.3 build. Run `Build-MSI.ps1` and send me the PowerShell output. I'll help resolve any build errors before we make changes to your working IIS deployment.

‹/›

You said:

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit> powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
Restore succeeded with 2 warning(s) in 1.1s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.

```
Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability,
https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.
Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability,
https://github.com/advisories/GHSA-846f-f764-76mx
  TovarTech.AssetManager.Web net10.0 win-x64 succeeded with 2 warning(s) (4.1s) → publish\

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.
Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability,
https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.
Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability,
https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 4 warning(s) in 5.8s
WiX not found. Installing wix .NET tool to user profile...
You can invoke the tool using the following command: wix
Tool 'wix' (version '7.0.0') was successfully installed.
wix.exe : error WIX7015: You must accept the Open Source Maintenance Fee (OSMF) EULA to use
WiX Toolset v7. For instructions, see https://wixtoolset.org/osmf/
WiX build failed; review errors above.
At C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\Build-MSI.ps1:65
char:27
+ ... $LASTEXITCODE -ne 0) {throw 'WiX build failed; review errors above.'}
+                          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OperationStopped: (WiX build failed; review errors above.:String) [],
RuntimeException
    + FullyQualifiedErrorId : WiX build failed; review errors above.

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit>
```

ChatGPT said:

Excellent progress! Your output tells me that **our TovarTech Asset Manager 2.3 application compiled successfully**, but the MSI packaging process stopped because of a WiX Toolset licensing requirement.

The good news is that our C# application is building. We don't need to start over, and your existing IIS website hasn't been changed by this failed MSI build.

What your build results mean

Component	Result
.NET 10 application	✓ Build succeeded
Windows x64 publishing	✓ Succeeded
WiX Toolset installation	✓ Version 7 installed
MSI creation	✗ Blocked by WiX license acceptance
Security dependencies	⚠ Two vulnerability advisories

There are two things I want us to address before proceeding.

1. Resolve the WiX Toolset licensing requirement

WiX 7 is asking you to accept its Open Source Maintenance Fee (OSMF) license agreement.

This is a licensing decision, not a programming error.

Please review the official instructions:

WiX Toolset OSMF licensing information ↗

I would not recommend automatically accepting the agreement in our build script without reviewing its terms.

We can either use WiX 7 under its applicable licensing terms or investigate an alternative installer-building approach.

2. Address the security warnings

Your application references `System.Security.Cryptography.Pkcs` version `9.0.4`, which NuGet reports as affected by moderate- and high-severity vulnerabilities.

I want to correct that before we distribute our installer.

First, run this command in your project folder:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
```

This will help us determine whether the vulnerable dependency is directly referenced or introduced through another package.

We should update the affected dependency to a patched, compatible version and rebuild rather than suppressing the warnings.

The quickest way to continue

After reviewing the WiX documentation, I recommend staying with WiX 7 for now. Its current licensing terms allow use below the specified revenue threshold without a maintenance fee, although you must still review and accept the agreement. FireGiant Docs +1

If you agree to the terms, open PowerShell and run:

```
wix eula accept wix7
```

Then, from your existing build folder, run:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

WiX documents the explicit license-acceptance mechanism for version 7. GitHub

If successful, we should get our Version 2.3 MSI.

Please don't install the MSI over your working IIS deployment yet. I want to verify the installer contents and address the security warnings before we modify the functioning website.

Send me the results of the second build and the vulnerable-package command. I'll use those results to guide our next corrections.

You said:

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit> powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
Restore succeeded with 2 warning(s) in 0.8s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package 'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package 'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability, https://github.com/advisories/GHSA-846f-f764-76mx
TovarTech.AssetManager.Web net10.0 win-x64 succeeded with 2 warning(s) (0.9s) → publish\

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package 'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package 'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability, https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 4 warning(s) in 2.4s
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(45) : error WIX0091: Duplicate Directory with identifier 'D00143AFD632560A05A30' found. Access modifiers (global, library, file, section) cannot prevent these conflicts. Ensure all your identifiers of a given type (Directory, File, etc.) are unique.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(46) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(47) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(45) : error WIX0091: Duplicate Directory with identifier 'D53C3F0CFE25AEDF4F0B7' found. Access modifiers (global, library, file, section) cannot prevent these conflicts. Ensure all your identifiers of a given type (Directory, File, etc.) are unique.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(46) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(47) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(48) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(49) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(50) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(51) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(49) : error WIX0091: Duplicate Directory with identifier 'DAA8BF8C40BAC354624FD' found. Access modifiers (global, library, file, section) cannot prevent these conflicts. Ensure all your identifiers of a given type (Directory, File, etc.) are unique.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(50

```
) : error WIX0092: Location of symbol related to previous error.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\installer.generated.wxs(51
) : error WIX0092: Location of symbol related to previous error.
WiX build failed; review errors above.
At C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit\Build-MSI.ps1:65
char:27
+ ... $LASTEXITCODE -ne 0) {throw 'WiX build failed; review errors above.'}
+                           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OperationStopped: (WiX build failed; review errors above.:String) [],
RuntimeException
    + FullyQualifiedErrorId : WiX build failed; review errors above.

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Build_Kit>
```

ChatGPT said:

Great progress! We've cleared the WiX licensing issue, and our .NET 10 application is still compiling successfully.

The new error is in **the MSI installer-generation script I provided**, not in your Razor Pages application or SQL Server.

What went wrong

WiX is reporting:

```
WIX0091: Duplicate Directory with identifier ...
```

Our script generated multiple directory definitions with the same identifier. WiX requires every directory identifier to be unique.

I recommend correcting the generator rather than manually editing `installer.generated.wxs`, because that file will be recreated each time you build.

We also still need to address the `System.Security.Cryptography.Pkcs` security warnings before distributing the installer.

Your existing IIS website should remain untouched by this failed build. There's no reason to uninstall it or change your database.

I've corrected the MSI generator

I inspected the original `Build-MSI.ps1` and found the cause: it created the same directory definition repeatedly for different published files.

I've revised the script so that each directory is defined once, and the installer components reference those directories.

TovarTech Asset Manager 2.3 — MSI Generator Fix
Updated PowerShell build script • Original application source • Fix notes

Download Corrected Version 2.3 Build Kit ↗

Step 1 — Build again

Extract the corrected ZIP into a new folder, then open PowerShell:

```
cd C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_MSI_Fixed_Generator

powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

Adjust the folder name if Windows extracts it differently.

Step 2 — Check the installer

If the build succeeds, run:

```
Get-Item .\output\*.msi |
    Select-Object Name, Length, LastWriteTime
```

The expected output is:

```
TovarTech_Asset_Manager_Web_2.3.0_x64.msi
```

Step 3 — Check the vulnerable dependency

Please also run:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
```

I want to use the results to identify the correct dependency update before installation.

Important: I corrected the generator, but I haven't been able to test the WiX build on Windows. This MSI also stages the application files; it does not yet automatically configure IIS.

For now, **don't uninstall or overwrite your working Version 2.2 website**. Once we have a successful MSI build and have resolved the security warnings, we'll proceed with a controlled upgrade that preserves your existing SQL database.

We're making progress toward the installer we originally envisioned: one package that installs TovarTech Asset Manager, configures IIS, and supports future upgrades.

You said:

when i try to download it says virus detected in the download file

ChatGPT said:

That warning is important. **Please don't bypass Windows Defender, disable antivirus protection, or whitelist the ZIP file.**

Because our Version 2.3 package contains PowerShell scripts that build and deploy an application, security software may flag it. However, we shouldn't assume it's a false positive.

The safest approach is to keep your existing IIS installation running and inspect the corrected build script separately.

The good news is that **you don't need to download another complete application package to fix the WiX duplicate-directory error.** We can work directly with the original files already on your laptop.

Here's how we'll proceed safely

I checked the contents of the ZIP I generated. It includes PowerShell deployment scripts, C# source files, and a PNG logo. I can't determine from that inspection alone why your antivirus flagged it.

Instead of downloading another ZIP, we can fix your existing `Build-MSI.ps1` directly.

Open the original script in VS Code:

```
code .\Build-MSI.ps1
```

Find this section:

```
$components=New-Object System.Collections.Generic.List[string]
$refs=New-Object System.Collections.Generic.List[string]
```

The problem is in the directory-generation loop immediately below it. We need to generate each WiX directory only once, rather than repeating it for every published file.

I can give you the complete corrected PowerShell script directly in our conversation so you can inspect and paste it into VS Code without downloading an executable or archive.

One question first: Was the virus warning displayed by Chrome/Edge during the download, or by Windows Defender after the file was downloaded?

That will help us determine whether we should also review a Windows Security detection report before proceeding.

You said:

lets just download the previous file please just provide a new download file with no virus

Worked for 14s

ChatGPT said:

I've created a new, simplified download package for **TovarTech Asset Manager Version 2.3**.

TovarTech Asset Manager 2.3
Source Code + Corrected MSI Build Script
Download TovarTech Asset Manager 2.3

This package contains the application source code and corrected MSI generator. I've excluded the IIS deployment scripts and compiled executables to simplify the download.

Please note: I verified the ZIP's integrity, but I cannot guarantee that antivirus software won't flag it. If you receive another security warning, don't bypass it.

Once downloaded, extract the files and run:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

The existing IIS application and SQL Server database should remain unchanged during this build process. We will address the outstanding dependency security warnings before deploying the new installer.

You said:

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(10) : warning WIX1070: The short file name 'e6mlzjqc.dll' was generated for multiple files that may be installed to the same directory. This could be due to conflicting long file names specified by the File/@Name attribute. If that is the case, please resolve the conflict in those attributes. Otherwise, please manually set the File/@ShortName attribute on the conflicting row to fix the collision. If one of the colliding files was added via a patch, that short file name should be specified manually to avoid disturbing the original short file name. C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w

xs(11) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(12) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(13) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(14) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(15) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(16) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(30) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(31) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(32) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(44) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(53) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(54) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
MSI built:
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi
IMPORTANT: This MSI stages application files. IIS deployment is a separate administrator step (see README).
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>

ChatGPT said:

Excellent! TovarTech Asset Manager 2.3 has successfully generated its first MSI installer!

Your PowerShell output confirms:

```
MSI built:
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi
```

We've resolved the previous WiX directory errors and completed the MSI build process.

Understanding the remaining warnings

The `WIX1070` and `WIX1071` warnings indicate that WiX generated duplicate short filenames for certain application files.

These warnings did not prevent the MSI from being created, but **we should resolve them before treating this as a production-ready installer**.

Component	Status
.NET 10 compilation	✓ Successful
WiX MSI generation	✓ Successful
Duplicate directory errors	✓ Resolved
Short filename warnings	⚠ Needs correction
NuGet security warnings	⚠ Needs verification
IIS deployment	Not yet performed

Next step: Verify the MSI without disturbing IIS

Before installation, please run:

```
Get-Item ".\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi" |
    Select-Object Name, Length, LastWriteTime
```

Then check the dependency vulnerabilities:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
```

Finally, verify the MSI's digital signature:

```
Get-AuthenticodeSignature ".\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi" |
    Select-Object Status, StatusMessage
```

An unsigned locally generated MSI may report `NotSigned`. That alone doesn't prove the installer is malicious, but it means Windows cannot verify its publisher through a code-signing certificate.

Do not install the MSI yet. We should fix the filename collisions, resolve the vulnerable dependency, and inspect the package contents first.

Once those checks pass, we'll test installation and upgrade behavior while preserving your existing IIS website and SQL Server database.

Our goal remains a professional installer that lets you deploy future TovarTech Asset Manager versions without manually rebuilding the application or risking existing asset records.

You said:

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
Restore succeeded with 2 warning(s) in 0.7s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package 'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability,

https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package 'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability, https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 2 warning(s) in 1.2s

The following sources were used:
https://api.nuget.org/v3/index.json
C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

Project `TovarTech.AssetManager.Web` has the following vulnerable packages
[net10.0]:
Transitive Package Resolved Severity Advisory URL
> System.Security.Cryptography.Pkcs 9.0.4 High https://github.com/advisories/GHSA-846f-f764-76mx
Moderate https://github.com/advisories/GHSA-2fmv-rv2c-59x7

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> Get-AuthenticodeSignature ".\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi" |
>> Select-Object Status, StatusMessage

Status StatusMessage
------ -------------
NotSigned The file C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi is not digitally signed. You cannot run this script on the current system. For more information ...

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only

ChatGPT said:

Excellent. These results give us a clear picture of what remains before we install **TovarTech Asset Manager 2.3**.

The MSI was generated successfully, but we've identified two issues that need attention.

1. Security vulnerability — confirmed

Your report identifies a vulnerable transitive dependency:

Property	Result
Package	`System.Security.Cryptography.Pkcs`
Installed version	9.0.4
Highest reported severity	High
Dependency type	Transitive
Action	Update dependency

A *transitive dependency* means another NuGet package is bringing this library into our project.

Let's identify which package is responsible.

Run:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --include-transitive
```

Then check whether NuGet identifies newer package versions:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --outdated
```

We can use those results to select a compatible patched version rather than guessing.

2. MSI digital signature — not signed

Your second result:

```
Status       StatusMessage
------       -------------
NotSigned    The file is not digitally signed
```

This is expected for an MSI we've built locally without a code-signing certificate.

It doesn't automatically mean the MSI contains malware.

For our development environment, signing isn't strictly required to test an installer. However, I recommend signing future installers before distributing them to other users.

We should also resolve the previously reported WiX short-filename collisions before testing this MSI.

3. What I recommend next

Our immediate goal is to produce a clean Version 2.3 build:

TovarTech Asset Manager 2.3 Validation pending

- ✓ Application compiles successfully
- ✓ MSI file generated
- ○ Resolve vulnerable dependency
- ○ Resolve WiX filename collisions
- ○ Validate MSI installation and IIS deployment

Please send me the output of the two `dotnet list` commands above. I'll use it to help identify the dependency we need to update.

For now, leave your existing IIS application at `http://localhost:8080` running. We can continue correcting the MSI build without changing your SQL Server database or disrupting the working application.

You said:

```
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list
.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --include-transitive
Restore succeeded with 2 warning(s) in 0.4s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability,
https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability,
https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 2 warning(s) in 0.9s
Project 'TovarTech.AssetManager.Web' has the following package references
  [net10.0]:
  Top-level Package             Requested   Resolved
  > Microsoft.Data.SqlClient    6.1.3       6.1.3

  Transitive Package                          Resolved
  > Azure.Core                                1.47.1
  > Azure.Identity                            1.14.2
  > Microsoft.Bcl.AsyncInterfaces             8.0.0
  > Microsoft.Bcl.Cryptography                9.0.4
  > Microsoft.Data.SqlClient.SNI.runtime      6.0.2
  > Microsoft.Identity.Client                 4.73.1
  > Microsoft.Identity.Client.Extensions.Msal 4.73.1
  > Microsoft.IdentityModel.Abstractions      7.7.1
  > Microsoft.IdentityModel.JsonWebTokens     7.7.1
  > Microsoft.IdentityModel.Logging           7.7.1
  > Microsoft.IdentityModel.Protocols         7.7.1
  > Microsoft.IdentityModel.Protocols.OpenIdConnect 7.7.1
  > Microsoft.IdentityModel.Tokens            7.7.1
  > Microsoft.SqlServer.Server                1.0.0
  > System.ClientModel                        1.5.1
  > System.Configuration.ConfigurationManager 9.0.4
  > System.IdentityModel.Tokens.Jwt           7.7.1
  > System.Memory.Data                        8.0.1
  > System.Security.Cryptography.Pkcs         9.0.4
  > System.Security.Cryptography.ProtectedData 9.0.4

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list
.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --outdated
Restore succeeded with 2 warning(s) in 0.4s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known moderate severity vulnerability,
https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package
'System.Security.Cryptography.Pkcs' 9.0.4 has a known high severity vulnerability,
https://github.com/advisories/GHSA-846f-f764-76mx
```

```
Build succeeded with 2 warning(s) in 1.0s

The following sources were used:
   https://api.nuget.org/v3/index.json
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

Project TovarTech.AssetManager.Web has the following updates to its packages
   [net10.0]:
   Top-level Package             Requested   Resolved   Latest
   > Microsoft.Data.SqlClient    6.1.3       6.1.3      7.1.1

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>
```

ChatGPT said:

Excellent! We now have the information needed to address the security vulnerability in **TovarTech Asset Manager 2.3**.

Your results confirm that our application uses:

- **.NET 10**
- **Microsoft.Data.SqlClient 6.1.3** as its only direct NuGet dependency
- **System.Security.Cryptography.Pkcs 9.0.4** as a vulnerable transitive dependency

NuGet also reports that `Microsoft.Data.SqlClient 7.1.1` is available.

Step 1 — Update our SQL Server driver

I recommend testing the upgrade to `Microsoft.Data.SqlClient 7.1.1` first. This may also update the dependencies responsible for the security warning.

Open PowerShell in your existing project folder and execute:

```
dotnet add .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package Microsoft.Data.SqlClient --version 7.1.1
```

This changes the project dependency, not your SQL Server database.

Step 2 — Restore and check vulnerabilities

Run:

```
dotnet restore .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj
```

Then:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
```

Our goal is to see:

```
The given project has no vulnerable packages given the current sources.
```

If the vulnerability remains, we'll inspect the updated dependency tree and select a patched version of the affected package. We shouldn't assume upgrading SqlClient alone will resolve it.

Step 3 — Rebuild our application

If the vulnerability check is clean, execute:

```
dotnet build .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj -c Release
```

Then generate the MSI again:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

Remember that we still need to resolve the WiX short-filename collision warnings before installing the MSI.

Step 4 — Verify the result

Test	Expected result
NuGet restore	Successful
Vulnerability audit	No known vulnerable packages
.NET build	Successful
MSI generation	Successful
WiX short-name warnings	Must be resolved
Existing IIS application	Remains unchanged

One important consideration: a major-version SqlClient upgrade can change connection behavior. Before deploying it, we'll test connectivity to your `noedell\DELLSQL` instance and verify that existing assets can still be read and updated.

Let's complete Steps 1 and 2 first. Send me the results of the vulnerability check after upgrading SqlClient to 7.1.1, and we'll determine whether the security issue is resolved before rebuilding the installer.

You said:

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
Restore succeeded with 2 warning(s) in 0.4s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package 'System.Security.Cryptography.Pkcs' 9.0.18 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package 'System.Security.Cryptography.Pkcs' 9.0.18 has a known high severity vulnerability, https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 2 warning(s) in 0.9s

```
The following sources were used:
   https://api.nuget.org/v3/index.json
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

Project TovarTech.AssetManager.Web has the following vulnerable packages
   [net10.0]:
   Transitive Package                    Resolved   Severity   Advisory URL
   > System.Security.Cryptography.Pkcs   9.0.18     High
https://github.com/advisories/GHSA-846f-f764-76mx
                                         Moderate   https://github.com/advisories/GHSA-2fmv-rv2c-
59x7

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>
```

ChatGPT said:

Your latest results show that we've made progress, but **the security vulnerability is still present**.

The affected package changed from `System.Security.Cryptography.Pkcs 9.0.4` to `9.0.18`, but NuGet still reports both high- and moderate-severity advisories.

The next step is to move this dependency to a patched version compatible with .NET 10.

Step 1 — Try updating the vulnerable package

Because we're developing TovarTech Asset Manager using .NET 10, let's try the current 10.0 servicing line rather than continuing with the vulnerable 9.0 releases.

In your existing PowerShell window, run:

```
dotnet add .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package System.Security.Cryptography.Pkcs
--version 10.0.1
```

This adds an explicit package reference that can override the vulnerable transitive version, provided NuGet resolves it without conflicts.

Step 2 — Restore and audit

```
dotnet restore .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj
```

Then:

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
```

We're looking for **no reported vulnerable packages**. If `10.0.1` is also flagged, or NuGet reports compatibility errors, don't suppress the warnings; we'll select a newer patched release.

Step 3 — Confirm the resolved versions

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --include-transitive
```

Confirm that `System.Security.Cryptography.Pkcs` now resolves to the selected version and that there are no downgrade warnings.

Step 4 — Build the application

Only after the vulnerability audit is clean, run:

```
dotnet build .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj -c Release
```

Do not install the MSI yet. We still need to correct the WiX short-filename collision warnings and validate the upgraded SQL client against your existing SQL Server Express database.

Send me the output from Step 2. Once we get a clean vulnerability report, we'll tackle the final MSI packaging issue.

You said:

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
Restore succeeded with 2 warning(s) in 0.5s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package 'System.Security.Cryptography.Pkcs' 10.0.1 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package 'System.Security.Cryptography.Pkcs' 10.0.1 has a known high severity vulnerability, https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 2 warning(s) in 1.2s

The following sources were used:
  https://api.nuget.org/v3/index.json
  C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

Project `TovarTech.AssetManager.Web` has the following vulnerable packages
  [net10.0]:
  Top-level Package Requested Resolved Severity Advisory URL
  > System.Security.Cryptography.Pkcs 10.0.1 10.0.1 High https://github.com/advisories/GHSA-846f-f764-76mx
                                                  Moderate https://github.com/advisories/GHSA-2fmv-rv2c-59x7

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --include-transitive
Restore succeeded with 2 warning(s) in 0.4s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package 'System.Security.Cryptography.Pkcs' 10.0.1 has a known moderate severity vulnerability, https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package 'System.Security.Cryptography.Pkcs' 10.0.1 has a known high severity vulnerability, https://github.com/advisories/GHSA-846f-f764-76mx

```
Build succeeded with 2 warning(s) in 0.9s
Project 'TovarTech.AssetManager.Web' has the following package references
   [net10.0]:
   Top-level Package                      Requested   Resolved
   > Microsoft.Data.SqlClient             7.1.1       7.1.1
   > System.Security.Cryptography.Pkcs    10.0.1      10.0.1

   Transitive Package                                 Resolved
   > Microsoft.Bcl.Cryptography                       9.0.18
   > Microsoft.Data.SqlClient.Extensions.Abstractions 7.1.1
   > Microsoft.Data.SqlClient.Internal.Logging        7.1.1
   > Microsoft.Data.SqlClient.SNI.runtime             7.1.0
   > Microsoft.IdentityModel.Abstractions             8.16.0
   > Microsoft.IdentityModel.JsonWebTokens            8.16.0
   > Microsoft.IdentityModel.Logging                  8.16.0
   > Microsoft.IdentityModel.Protocols                8.16.0
   > Microsoft.IdentityModel.Protocols.OpenIdConnect  8.16.0
   > Microsoft.IdentityModel.Tokens                   8.16.0
   > Microsoft.SqlServer.Server                       1.0.0
   > System.Configuration.ConfigurationManager        9.0.18
   > System.IdentityModel.Tokens.Jwt                  8.16.0
   > System.Security.Cryptography.ProtectedData       9.0.18

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet build
.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj -c Release
Restore succeeded with 2 warning(s) in 0.5s

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package
'System.Security.Cryptography.Pkcs' 10.0.1 has a known moderate severity vulnerability,
https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package
'System.Security.Cryptography.Pkcs' 10.0.1 has a known high severity vulnerability,
https://github.com/advisories/GHSA-846f-f764-76mx
  TovarTech.AssetManager.Web net10.0 succeeded with 2 warning(s) (1.8s) →
TovarTech.AssetManager.Web\bin\Release\net10.0\TovarTech.AssetManager.Web.dll

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1902: Package
'System.Security.Cryptography.Pkcs' 10.0.1 has a known moderate severity vulnerability,
https://github.com/advisories/GHSA-2fmv-rv2c-59x7

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj : warning NU1903: Package
'System.Security.Cryptography.Pkcs' 10.0.1 has a known high severity vulnerability,
https://github.com/advisories/GHSA-846f-f764-76mx

Build succeeded with 4 warning(s) in 3.1s
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>
```

ChatGPT said:

Excellent. Your latest results confirm that **TovarTech Asset Manager 2.3 compiles successfully with .NET 10 and Microsoft.Data.SqlClient 7.1.1**.

However, our security audit still identifies a high-severity vulnerability in `System.Security.Cryptography.Pkcs 10.0.1`.

The important distinction is that a successful build does not mean the dependencies are secure.

What we've confirmed

Component	Current version	Status
.NET	10.0	✓ Build successful
Microsoft.Data.SqlClient	7.1.1	✓ Updated
System.Security.Cryptography.Pkcs	10.0.1	⚠ Vulnerable
MSI generator	2.3	⚠ Short-name warnings remain
Existing IIS website	Version 2.2	No deployment changes made

Next step — Find a patched PKCS version

Rather than guessing another package version, let's ask NuGet which versions are available.

From your current PowerShell directory, run:

```
dotnet package search System.Security.Cryptography.Pkcs --exact-match --format json
```

This helps identify the available package release. We then need to check whether that release addresses both advisories.

You can also review the security advisories directly:

- GHSA-846f-f764-76mx ↗
- GHSA-2fmv-rv2c-59x7 ↗

Once we identify an appropriate patched version, we'll update the explicit reference and rerun the vulnerability audit.

Why the SQL driver upgrade was useful

Upgrading to SqlClient 7.1.1 simplified the dependency tree considerably. The older Azure-related packages are no longer present in the output you shared.

That gives us a smaller set of dependencies to maintain, although we still need to verify database connectivity after the upgrade.

Our release criteria

Before installing Version 2.3, I want us to achieve three things: a clean vulnerability audit, an MSI build without short-filename collisions, and successful SQL Server read/write tests.

Please send me the output of the `dotnet package search` command. We'll use that information to select the next PKCS version instead of continuing to guess.

You said:

```
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet package
search System.Security.Cryptography.Pkcs --exact-match --format json
{
  "version": 2,
  "problems": [],
  "searchResult": [
    {
      "sourceName": "Microsoft Visual Studio Offline Packages",
      "packages": []
    },
    {
      "sourceName": "nuget.org",
      "packages": [
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.0.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.3.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.4.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.5.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.5.1"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.5.2"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.6.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "4.7.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "5.0.0"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
          "version": "5.0.1"
        },
        {
          "id": "System.Security.Cryptography.Pkcs",
```

```
  "version": "6.0.0"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "6.0.1"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "6.0.2"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "6.0.3"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "6.0.4"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "6.0.5"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "7.0.0"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "7.0.1"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "7.0.2"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "7.0.3"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "8.0.0"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "8.0.1"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "8.0.2"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.0"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.1"
```

```
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.2"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.3"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.4"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.5"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.6"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.7"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.8"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.9"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.10"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.11"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.12"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.13"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.14"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.15"
},
```

```
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.16"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.17"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.18"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.19"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.20"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "9.0.21"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.0"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.1"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.2"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.3"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.4"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.5"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.6"
},
{
  "id": "System.Security.Cryptography.Pkcs",
  "version": "10.0.7"
},
{
```

```
        "id": "System.Security.Cryptography.Pkcs",
        "version": "10.0.8"
      },
      {
        "id": "System.Security.Cryptography.Pkcs",
        "version": "10.0.9"
      },
      {
        "id": "System.Security.Cryptography.Pkcs",
        "version": "10.0.10"
      },
      {
        "id": "System.Security.Cryptography.Pkcs",
        "version": "10.0.11"
      },
      {
        "id": "System.Security.Cryptography.Pkcs",
        "version": "10.0.12"
      },
      {
        "id": "System.Security.Cryptography.Pkcs",
        "version": "10.0.13"
      }
    ]
  }
 ]
}
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>
```


ChatGPT said:

Excellent! NuGet confirms that **System.Security.Cryptography.Pkcs 10.0.13** is available. We're currently using version `10.0.1`, which has two reported security vulnerabilities.

I recommend testing version `10.0.13` next. It's the newest version in your NuGet results, although we still need to verify that it fixes both advisories.

Step 1 — Update the vulnerable package

From your current PowerShell directory, execute:

```
dotnet add .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package System.Security.Cryptography.Pkcs --version 10.0.13
```

This updates the package reference in our project. It will not modify your existing SQL Server database or IIS deployment.

Step 2 — Check for vulnerabilities

Run:

```
dotnet restore .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj
```

```
dotnet list .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --include-transitive
```

We're looking for a report indicating that no vulnerable packages were found.

Step 3 — Rebuild

If the security audit is clean, run:

```
dotnet build .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj -c Release
```

Then:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

Our Version 2.3 progress

Component	Status
ASP.NET Core / .NET 10	✓ Working
SQL Server driver 7.1.1	✓ Compiles
PKCS security update	⏱ Testing 10.0.13
MSI generation	✓ Previously successful
WiX filename collisions	⚠ Still needs correction
IIS deployment	Not started

Please send me the vulnerability report after updating to 10.0.13. If it's clean, we'll address the remaining WiX filename warnings and prepare to test our MSI installer safely.

You said:

PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet add .\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package System.Security.Cryptography.Pkcs --version 10.0.13
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'System.Security.Cryptography.Pkcs' into project '.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj'.
info : Restoring packages for C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj...
info : CACHE https://api.nuget.org/v3-flatcontainer/system.security.cryptography.pkcs/index.json
info : GET https://api.nuget.org/v3-flatcontainer/system.security.cryptography.pkcs/10.0.13/system.security.cryptography.pkcs.10.0.13.nupkg
info : OK https://api.nuget.org/v3-flatcontainer/system.security.cryptography.pkcs/10.0.13/system.security.cryptography.pkcs.10.0.13.nupkg 57ms
info : Installed System.Security.Cryptography.Pkcs 10.0.13 from https://api.nuget.org/v3/index.json to C:\Users\noe\.nuget\packages\system.security.cryptography.pkcs\10.0.13 with content hash a+EovgC1fBnwdlQ6cn3j4JFBfKqiFaFvoAhjisOoSvazLAcIznaBzPQP0oYnvccmeXfqUH1KebVZOxSeH6rMaw==.

```
info :   CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info :   CACHE https://api.nuget.org/v3-vulnerabilities/2026.10.19.23.33.13/vulnerability.base.json
info :   CACHE https://api.nuget.org/v3-
vulnerabilities/2026.10.19.23.33.13/2026.10.21.23.33.24/vulnerability.update.json
info : Package 'System.Security.Cryptography.Pkcs' is compatible with all the specified
frameworks in project '.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj'.
info : PackageReference for package 'System.Security.Cryptography.Pkcs' version '10.0.13'
updated in file
'C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj'.
info : Writing assets file to disk. Path:
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\obj\project.assets.json
log  : Restored
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\TovarTech.AssetMan
ager.Web\TovarTech.AssetManager.Web.csproj (in 4.09 sec).
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet restore
.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj
Restore complete (0.6s)

Build succeeded in 1.6s
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> dotnet list
.\TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj package --vulnerable --
include-transitive
Restore complete (0.6s)

Build succeeded in 1.3s

The following sources were used:
   https://api.nuget.org/v3/index.json
   C:\Program Files (x86)\Microsoft SDKs\NuGetPackages\

The given project `TovarTech.AssetManager.Web` has no vulnerable packages given the current
sources.
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only> powershell -
ExecutionPolicy Bypass -File .\Build-MSI.ps1
Restore complete (0.8s)
  TovarTech.AssetManager.Web net10.0 win-x64 succeeded (1.8s) → publish\

Build succeeded in 3.3s
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(8) : warning WIX1070: The short file name 'e6mlzjqc.dll' was generated for multiple files that
may be installed to the same directory. This could be due to conflicting long file names specified
by the File/@Name attribute. If that is the case, please resolve the conflict in those attributes.
Otherwise, please manually set the File/@ShortName attribute on the conflicting row to fix the
collision. If one of the colliding files was added via a patch, that short file name should be
specified manually to avoid disturbing the original short file name.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(9) : warning WIX1071: The location of a conflicting generated short file name related to the
previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(10) : warning WIX1071: The location of a conflicting generated short file name related to the
previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.w
xs(11) : warning WIX1071: The location of a conflicting generated short file name related to the
previous warning.
```

C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(12) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(13) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(14) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(27) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(28) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(29) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(39) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(48) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\installer.generated.wxs(49) : warning WIX1071: The location of a conflicting generated short file name related to the previous warning.
MSI built:
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi
IMPORTANT: This MSI stages application files. IIS deployment is a separate administrator step (see README).
PS C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only>

ChatGPT said:

Excellent! We've reached a major milestone with TovarTech Asset Manager 2.3.

Your latest PowerShell output confirms that the security vulnerability warnings are gone and our MSI installer builds successfully.

Version 2.3 build results

TovarTech Asset Manager 2.3 — Build successful

.NET 10 application	✓ Successful
SQL Client 7.1.1	✓ Compiles
PKCS 10.0.13	✓ Updated
NuGet vulnerability audit	✓ Clean
MSI generation	✓ Successful
WiX short-name collisions	⚠ Unresolved
IIS deployment	Not yet tested

Your vulnerability audit now reports:

```
The given project `TovarTech.AssetManager.Web`
has no vulnerable packages given the current sources.
```

That's exactly the result we wanted. It means NuGet found no known vulnerabilities in the resolved packages using your configured sources and current advisory data.

One remaining installer issue

The WiX warnings still concern me:

```
WIX1070: The short file name 'e6mlzjqc.dll' was generated for multiple files
```

Although WiX generated the MSI, we should correct these collisions before installing it. Otherwise, we risk files being installed incorrectly.

The next step is to inspect the generated WiX file and determine whether our build script is assigning incorrect filenames or directories.

Please run this PowerShell command

From your current project directory:

```
Get-Content .\installer.generated.wxs |
    Select-Object -First 65
```

This will show the beginning of the generated installer definition, including the file entries responsible for the warnings.

Paste the output here. I'll examine it and give you the specific correction to make in `Build-MSI.ps1`, so we can rebuild the MSI without downloading another ZIP.

After that, we'll validate the MSI contents and perform a controlled installation test while preserving your working IIS website and SQL Server database.

You said:

Pasted text(202610...
Document

ChatGPT said:

Excellent! I reviewed the WiX installer output you provided, and **we've found the cause of the remaining warnings**.

The problem is in our MSI generator, not the .NET application.

What's causing the warnings?

Your generated installer contains entries such as:

```
<DirectoryRef Id="INSTALLFOLDER">
  <Component Id="FileComponent1">
    <File
      Id="PublishedFile1"
      Source="...\publish\cs\Microsoft.Data.SqlClient.resources.dll"
```

```
      KeyPath="yes" />
  </Component>
</DirectoryRef>
```

It repeats the same structure for `de`, `es`, `fr`, `ja`, and other language folders.

The problem: Every file is assigned to `INSTALLFOLDER`, regardless of its original subdirectory. Pasted text.txt

That means WiX is trying to install multiple files named `Microsoft.Data.SqlClient.resources.dll` into the same destination directory.

The same issue affects your CSS, JavaScript, and image files. Pasted text.txt

The correct solution

We need to modify `Build-MSI.ps1` so that it preserves the published application's directory structure.

For example:

CURRENT — INCORRECT

```
INSTALLFOLDER
├── Microsoft.Data.SqlClient.resources.dll
├── Microsoft.Data.SqlClient.resources.dll
├── site.css
└── dashboard.js
```

REQUIRED — CORRECT

```
INSTALLFOLDER
├── cs
│   └── Microsoft.Data.SqlClient.resources.dll
├── de
│   └── Microsoft.Data.SqlClient.resources.dll
├── es
│   └── Microsoft.Data.SqlClient.resources.dll
└── wwwroot
    ├── css
    │   └── site.css
    ├── js
    │   └── dashboard.js
    └── images
        └── tovartech-logo.png
```

This should eliminate the filename collisions while preserving the directory structure ASP.NET Core needs.

Let's correct the actual build script

Open your existing script:

```
code .\Build-MSI.ps1
```

The generator needs to create one WiX `Directory` element per unique relative folder, with each `File` component referencing its proper destination. It must also preserve stable component GUIDs for future MSI upgrades.

I can provide the complete replacement script, but I want to avoid accidentally changing your working installer configuration.

Please paste or attach the contents of `Build-MSI.ps1` next. I'll revise that exact script, preserving your current MSI product information and build settings, so you can copy the correction directly into VS Code without another ZIP download.

We're very close to having a clean Version 2.3 installer.

You said:

```
[CmdletBinding()]
param()
$ErrorActionPreference='Stop'
$root=$PSScriptRoot
$project=Join-Path $root 'TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj'
$publish=Join-Path $root 'publish'
$output=Join-Path $root 'output'
New-Item -ItemType Directory -Force -Path $publish,$output | Out-Null
if (-not (Get-Command dotnet -ErrorAction SilentlyContinue)) {throw 'Install .NET 10 SDK.'}
& dotnet publish $project -c Release -r win-x64 --self-contained false -o $publish
if ($LASTEXITCODE -ne 0) {throw 'dotnet publish failed'}
$wix=Get-Command wix -ErrorAction SilentlyContinue
if (-not $wix) {
 Write-Host 'WiX not found. Installing wix .NET tool to user profile...'
 & dotnet tool install --global wix
 if ($LASTEXITCODE -ne 0) {throw 'WiX installation failed. Install WiX Toolset manually.'}
 $env:PATH += ";$env:USERPROFILE\.dotnet\tools"
}
if (-not (Get-Command wix -ErrorAction SilentlyContinue)) {throw 'WiX executable not found.
Reopen PowerShell and retry.'}
# Generate one component per published file, with stable deterministic GUIDs.
Add-Type -AssemblyName System.Security
function StableGuid([string]$text) {
 $bytes=[System.Text.Encoding]::UTF8.GetBytes($text.ToLowerInvariant())
 $hash=([System.Security.Cryptography.MD5]::Create()).ComputeHash($bytes)
 return New-Object Guid (,$hash)
}
function XmlEscape([string]$s) {return [System.Security.SecurityElement]::Escape($s)}
$files=Get-ChildItem $publish -File -Recurse | Sort-Object FullName
if (-not $files) {throw 'Publish folder is empty'}
# Define each directory exactly once, then reference it from file components.
# The previous generator repeated <Directory> definitions for every file.
$directories=New-Object System.Collections.Generic.List[string]
$seenDirs=New-Object 'System.Collections.Generic.HashSet[string]'
([System.StringComparer]::OrdinalIgnoreCase)
$components=New-Object System.Collections.Generic.List[string]
$refs=New-Object System.Collections.Generic.List[string]
$i=0
foreach ($file in $files) {
 $relative=$file.FullName.Substring($publish.Length+1)
 $segments=$relative -split '[\/]'
 $parent='INSTALLFOLDER'
 $parts=New-Object System.Collections.Generic.List[string]
 for($j=0;$j -lt $segments.Length-1;$j++) {
   $seg=$segments[$j]
   $parts.Add($seg)
   $dirPath=$parts -join '/'
   $id='D'+
([BitConverter]::ToString(([System.Security.Cryptography.MD5]::Create()).ComputeHash([Text.Enco
ding]::UTF8.GetBytes($dirPath.ToLowerInvariant()))).Replace('-','').Substring(0,20))
```

```
  if ($seenDirs.Add($dirPath)) {
    $directories.Add("<DirectoryRef Id="$parent"><Directory Id="$id" Name="$(XmlEscape
$seg)" /></DirectoryRef>")
  }
  $parent=$id
 }
 $cid="FileComponent$i"; $fid="PublishedFile$i";
$guid=StableGuid("TovarTechAssetManager2.3/$relative")
 $components.Add("<DirectoryRef Id="$parent"><Component Id="$cid" Guid="$guid">
<File Id="$fid" Source="$(XmlEscape $file.FullName)" KeyPath="yes" /></Component>
</DirectoryRef>")
 $refs.Add("<ComponentRef Id="$cid" />")
 $i++
}
# A stable product UpgradeCode allows future 2.x MSI major upgrades. Different from the
WinForms MSI.
$wxs=@"
<Wix xmlns="http://wixtoolset.org/schemas/v4/wxs">
 <Package Name="TovarTech Asset Manager Web" Manufacturer="TovarTech" Version="2.3.0"
UpgradeCode="5e0d9f2b-6b5a-4e71-9d61-9c6a2d86d7ac" Scope="perMachine">
  <MajorUpgrade DowngradeErrorMessage="A newer version is already installed." />
  <MediaTemplate EmbedCab="yes" />
  <StandardDirectory Id="ProgramFiles64Folder"><Directory Id="INSTALLFOLDER"
Name="TovarTech Asset Manager Web" /></StandardDirectory>
  $($directories -join "`n")
  $($components -join "`n")
  <Feature Id="MainFeature" Title="TovarTech Asset Manager Web" Level="1">$($refs -join "`n")
</Feature>
 </Package>
</Wix>
"@
$wxsPath=Join-Path $root 'installer.generated.wxs'
Set-Content -Path $wxsPath -Value $wxs -Encoding UTF8
$msi=Join-Path $output 'TovarTech_Asset_Manager_Web_2.3.0_x64.msi'
& wix build $wxsPath -arch x64 -out $msi
if ($LASTEXITCODE -ne 0) {throw 'WiX build failed; review errors above.'}
Write-Host "MSI built: $msi" -ForegroundColor Green
Write-Host 'IMPORTANT: This MSI stages application files. IIS deployment is a separate
administrator step (see README).'
```

ChatGPT said:

I've reviewed your complete `Build-MSI.ps1` script. We can now correct the installer generator without downloading another ZIP file.

The script you pasted already contains some directory-handling improvements, but I want to strengthen three areas:

1. **Preserve the complete directory structure** of the published ASP.NET Core application.
2. **Prevent duplicate directory definitions and filename collisions** in the WiX installer.
3. **Maintain stable component identifiers** so future MSI upgrades are more reliable.

We'll preserve your existing application version, MSI filename, and UpgradeCode.

Step 1 — Back up the current script

In your existing PowerShell window, run:

```
Copy-Item .\Build-MSI.ps1 .\Build-MSI.ps1.backup
```

Then open the script:

```
notepad .\Build-MSI.ps1
```

Replace its contents with the following corrected version.

Step 2 — Complete corrected Build-MSI.ps1

```
[CmdletBinding()]
param()

$ErrorActionPreference = 'Stop'

$root = $PSScriptRoot
$project = Join-Path $root 'TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj'
$publish = Join-Path $root 'publish'
$output = Join-Path $root 'output'

# Preserve the MSI identity across future upgrades.
$version = '2.3.0'
$upgradeCode = '5e0d9f2b-6b5a-4e71-9d61-9c6a2d86d7ac'

function XmlEscape([string]$value) {
    return [System.Security.SecurityElement]::Escape($value)
}

# Generate deterministic GUIDs from stable relative paths.
function StableGuid([string]$value) {
    $bytes = [System.Text.Encoding]::UTF8.GetBytes(
        $value.ToLowerInvariant()
    )

    $sha = [System.Security.Cryptography.SHA256]::Create()
    try {
        $hash = $sha.ComputeHash($bytes)
    }
    finally {
        $sha.Dispose()
    }

    $guidBytes = New-Object byte[] 16
    [Array]::Copy($hash, $guidBytes, 16)

    return [Guid]::new($guidBytes)
}

function StableId([string]$prefix, [string]$value) {
    $bytes = [System.Text.Encoding]::UTF8.GetBytes(
        $value.ToLowerInvariant()
    )

    $sha = [System.Security.Cryptography.SHA256]::Create()
    try {
```

```
        $hash = $sha.ComputeHash($bytes)
    }
    finally {
        $sha.Dispose()
    }

    $hex = [BitConverter]::ToString($hash).
        Replace('-', '').Substring(0, 24)

    return "$prefix$hex"
}

Write-Host ""
Write-Host "========================================" -ForegroundColor Cyan
Write-Host " TovarTech Asset Manager MSI Builder" -ForegroundColor Cyan
Write-Host " Version $version" -ForegroundColor Cyan
Write-Host "========================================" -ForegroundColor Cyan
Write-Host ""

if (-not (Get-Command dotnet -ErrorAction SilentlyContinue)) {
    throw 'The .NET SDK was not found.'
}

# Remove stale published files before rebuilding.
if (Test-Path $publish) {
    Remove-Item $publish -Recurse -Force
}

New-Item -ItemType Directory -Force -Path $publish, $output |
    Out-Null

Write-Host "Publishing .NET application..." -ForegroundColor Yellow

& dotnet publish $project `
    -c Release `
    -r win-x64 `
    --self-contained false `
    -o $publish

if ($LASTEXITCODE -ne 0) {
    throw 'dotnet publish failed.'
}

# Locate WiX Toolset.
if (-not (Get-Command wix -ErrorAction SilentlyContinue)) {
    $env:PATH += ";$env:USERPROFILE\.dotnet\tools"
}

if (-not (Get-Command wix -ErrorAction SilentlyContinue)) {
    throw 'WiX Toolset is not installed.'
}

Write-Host "Generating WiX installer definition..." -ForegroundColor Yellow

$files = @(
    Get-ChildItem -Path $publish -File -Recurse |
        Sort-Object FullName
```

```
)

if ($files.Count -eq 0) {
    throw 'The publish directory contains no files.'
}

# Store each directory exactly once.
$directoryIds = @{}
$directoryIds[''] = 'INSTALLFOLDER'

$directoryDefinitions =
    New-Object System.Collections.Generic.List[string]

$componentDefinitions =
    New-Object System.Collections.Generic.List[string]

$componentReferences =
    New-Object System.Collections.Generic.List[string]

foreach ($file in $files) {

    $relative = $file.FullName.Substring(
        $publish.Length + 1
    )

    $relative = $relative.Replace('\', '/')

    $segments = $relative.Split('/')

    $parentId = 'INSTALLFOLDER'
    $currentPath = ''

    # Create each required parent directory once.
    for ($i = 0; $i -lt ($segments.Length - 1); $i++) {

        $segment = $segments[$i]

        if ($currentPath -eq '') {
            $currentPath = $segment
        }
        else {
            $currentPath += '/' + $segment
        }

        if (-not $directoryIds.ContainsKey($currentPath)) {

            $directoryId = StableId 'DIR_' $currentPath

            $directoryIds[$currentPath] = $directoryId

            $safeName = XmlEscape $segment

            $directoryDefinitions.Add(
                "<DirectoryRef Id=`"$parentId`">" +
                "<Directory Id=`"$directoryId`" Name=`"$safeName`" />" +
                "</DirectoryRef>"
            )
```

```
            }

            $parentId = $directoryIds[$currentPath]
        }

        $componentId = StableId 'CMP_' $relative
        $fileId = StableId 'FIL_' $relative

        $componentGuid = StableGuid (
            "TovarTechAssetManagerWeb/" + $relative
        )

        $source = XmlEscape $file.FullName
        $fileName = XmlEscape $file.Name

        $componentDefinitions.Add(
            "<DirectoryRef Id=`"$parentId`">" +
            "<Component Id=`"$componentId`" Guid=`"$componentGuid`">" +
            "<File Id=`"$fileId`" Name=`"$fileName`" Source=`"$source`" KeyPath=`"yes`" />" +
            "</Component>" +
            "</DirectoryRef>"
        )

        $componentReferences.Add(
            "<ComponentRef Id=`"$componentId`" />"
        )
    }

    $directoryXml = $directoryDefinitions -join "`r`n"
    $componentXml = $componentDefinitions -join "`r`n"
    $referenceXml = $componentReferences -join "`r`n"

    $wxs = @"
    <Wix xmlns="http://wixtoolset.org/schemas/v4/wxs">
      <Package
          Name="TovarTech Asset Manager Web"
          Manufacturer="TovarTech"
          Version="$version"
          UpgradeCode="$upgradeCode"
          Scope="perMachine">

        <MajorUpgrade
            DowngradeErrorMessage="A newer version is already installed." />

        <MediaTemplate EmbedCab="yes" />

        <StandardDirectory Id="ProgramFiles64Folder">
          <Directory
              Id="INSTALLFOLDER"
              Name="TovarTech Asset Manager Web" />
        </StandardDirectory>

    $directoryXml

    $componentXml

        <Feature
```

```
        Id="MainFeature"
        Title="TovarTech Asset Manager Web"
        Level="1">

$referenceXml

    </Feature>
  </Package>
</Wix>
"@

$wxsPath = Join-Path $root 'installer.generated.wxs'

Set-Content -Path $wxsPath -Value $wxs -Encoding UTF8

Write-Host "WiX definition generated." -ForegroundColor Green
Write-Host "Files included: $($files.Count)" -ForegroundColor Cyan
Write-Host "Subdirectories: $($directoryIds.Count - 1)" -ForegroundColor Cyan

$msi = Join-Path $output (
    "TovarTech_Asset_Manager_Web_${version}_x64.msi"
)

# Delete the previous MSI to avoid mistaking it for a new build.
if (Test-Path $msi) {
    Remove-Item $msi -Force
}

Write-Host "Building MSI..." -ForegroundColor Yellow

& wix build $wxsPath -arch x64 -out $msi

if ($LASTEXITCODE -ne 0) {
    throw 'WiX build failed.'
}

if (-not (Test-Path $msi)) {
    throw 'MSI output was not created.'
}

Write-Host ""
Write-Host "========================================" -ForegroundColor Green
Write-Host " MSI BUILD COMPLETED" -ForegroundColor Green
Write-Host "========================================" -ForegroundColor Green

Write-Host "Installer: $msi" -ForegroundColor Cyan

Get-Item $msi |
    Select-Object Name, Length, LastWriteTime

Write-Host ""
Write-Host "IMPORTANT: This MSI installs application files only."
Write-Host "IIS configuration is a separate deployment step."
```

This version explicitly places each file into its corresponding WiX directory. It also uses deterministic identifiers based on relative paths rather than file enumeration order.

Step 3 — Rebuild Version 2.3

Save the file and return to PowerShell.

Run:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

If successful, the final output should resemble:

```
========================================
 MSI BUILD COMPLETED
========================================

Installer:
C:\Users\noe\Downloads\TovarTech_Asset_Manager_2_3_Source_Build_Only\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi
```

The exact file count and directory count will depend on your published application.

Step 4 — Verify that the folder structure is correct

After the build, run:

```
Select-String -Path .\installer.generated.wxs `
    -Pattern 'Directory Id=|DirectoryRef Id=' |
    Select-Object -First 25
```

We should see separate directory identifiers for the language folders and `wwwroot` subdirectories rather than every component referencing `INSTALLFOLDER`.

Where we stand

Our NuGet security audit is now clean, the application compiles successfully, and we have a specific correction for the WiX filename collisions.

One detail remains important: this revised generator is intended for a **fresh test installation**. Because it changes component GUID generation compared with earlier experimental MSI builds, we should not use it to upgrade an already-installed copy of the previous 2.3 MSI without first reviewing its installed component identity. Your existing IIS deployment is separate and should remain untouched.

Run the corrected script and send me the output. If the WiX warnings are gone, our next milestone will be validating the MSI installation, confirming that the application's folder structure is preserved, and then preparing the controlled IIS upgrade.

You said:

```
[CmdletBinding()]
param()
$ErrorActionPreference='Stop'
$root=$PSScriptRoot
$project=Join-Path $root 'TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj'
$publish=Join-Path $root 'publish'
$output=Join-Path $root 'output'
New-Item -ItemType Directory -Force -Path $publish,$output | Out-Null
```

```
if (-not (Get-Command dotnet -ErrorAction SilentlyContinue)) {throw 'Install .NET 10 SDK.'}
& dotnet publish $project -c Release -r win-x64 --self-contained false -o $publish
if ($LASTEXITCODE -ne 0) {throw 'dotnet publish failed'}
$wix=Get-Command wix -ErrorAction SilentlyContinue
if (-not $wix) {
 Write-Host 'WiX not found. Installing wix .NET tool to user profile...'
 & dotnet tool install --global wix
 if ($LASTEXITCODE -ne 0) {throw 'WiX installation failed. Install WiX Toolset manually.'}
 $env:PATH += ";$env:USERPROFILE\.dotnet\tools"
}
if (-not (Get-Command wix -ErrorAction SilentlyContinue)) {throw 'WiX executable not found.
Reopen PowerShell and retry.'}
# Generate one component per published file, with stable deterministic GUIDs.
Add-Type -AssemblyName System.Security
function StableGuid([string]$text) {
 $bytes=[System.Text.Encoding]::UTF8.GetBytes($text.ToLowerInvariant())
 $hash=([System.Security.Cryptography.MD5]::Create()).ComputeHash($bytes)
 return New-Object Guid (,$hash)
}
function XmlEscape([string]$s) {return [System.Security.SecurityElement]::Escape($s)}
$files=Get-ChildItem $publish -File -Recurse | Sort-Object FullName
if (-not $files) {throw 'Publish folder is empty'}
# Define each directory exactly once, then reference it from file components.
# The previous generator repeated <Directory> definitions for every file.
$directories=New-Object System.Collections.Generic.List[string]
$seenDirs=New-Object 'System.Collections.Generic.HashSet[string]'
([System.StringComparer]::OrdinalIgnoreCase)
$components=New-Object System.Collections.Generic.List[string]
$refs=New-Object System.Collections.Generic.List[string]
$i=0
foreach ($file in $files) {
 $relative=$file.FullName.Substring($publish.Length+1)
 $segments=$relative -split '[\/]'
 $parent='INSTALLFOLDER'
 $parts=New-Object System.Collections.Generic.List[string]
 for($j=0;$j -lt $segments.Length-1;$j++) {
   $seg=$segments[$j]
   $parts.Add($seg)
   $dirPath=$parts -join '/'
   $id='D'+
([BitConverter]::ToString(([System.Security.Cryptography.MD5]::Create()).ComputeHash([Text.Enco
ding]::UTF8.GetBytes($dirPath.ToLowerInvariant()))).Replace('-','').Substring(0,20))
   if ($seenDirs.Add($dirPath)) {
     $directories.Add("<DirectoryRef Id="$parent"><Directory Id="$id" Name="$(XmlEscape
$seg)" /></DirectoryRef>")
   }
   $parent=$id
 }
 $cid="FileComponent$i"; $fid="PublishedFile$i";
$guid=StableGuid("TovarTechAssetManager2.3/$relative")
 $components.Add("<DirectoryRef Id="$parent"><Component Id="$cid" Guid="$guid">
<File Id="$fid" Source="$(XmlEscape $file.FullName)" KeyPath="yes" /></Component>
</DirectoryRef>")
 $refs.Add("<ComponentRef Id="$cid" />")
 $i++
}
# A stable product UpgradeCode allows future 2.x MSI major upgrades. Different from the
```

```
WinForms MSI.
$wxs=@"
<Wix xmlns="http://wixtoolset.org/schemas/v4/wxs">
 <Package Name="TovarTech Asset Manager Web" Manufacturer="TovarTech" Version="2.3.0"
UpgradeCode="5e0d9f2b-6b5a-4e71-9d61-9c6a2d86d7ac" Scope="perMachine">
  <MajorUpgrade DowngradeErrorMessage="A newer version is already installed." />
  <MediaTemplate EmbedCab="yes" />
  <StandardDirectory Id="ProgramFiles64Folder"><Directory Id="INSTALLFOLDER"
Name="TovarTech Asset Manager Web" /></StandardDirectory>
  $($directories -join "`n")
  $($components -join "`n")
  <Feature Id="MainFeature" Title="TovarTech Asset Manager Web" Level="1">$($refs -join "`n")
</Feature>
 </Package>
</Wix>
"@
$wxsPath=Join-Path $root 'installer.generated.wxs'
Set-Content -Path $wxsPath -Value $wxs -Encoding UTF8
$msi=Join-Path $output 'TovarTech_Asset_Manager_Web_2.3.0_x64.msi'
& wix build $wxsPath -arch x64 -out $msi
if ($LASTEXITCODE -ne 0) {throw 'WiX build failed; review errors above.'}
Write-Host "MSI built: $msi" -ForegroundColor Green
Write-Host 'IMPORTANT: This MSI stages application files. IIS deployment is a separate
administrator step (see README).'
```

ChatGPT said:

I reviewed the `Build-MSI.ps1` script you pasted. This is still the **original version**, not the corrected version I provided previously.

The good news is that we only need to make one targeted change to fix the directory-generation problem.

Step 1 — Back up your script

Run:

```
Copy-Item .\Build-MSI.ps1 .\Build-MSI.ps1.backup -Force
notepad .\Build-MSI.ps1
```

Step 2 — Correct the directory-generation expression

Find this line:

```
$segments=$relative -split '[\/]'
```

Replace it with:

```
$segments = $relative -split '[\\/]'
```

Why this matters: Your original regular expression matches forward slashes but not Windows backslashes. Since Windows file paths use `\`, the script wasn't separating paths such as:

```
wwwroot\css\site.css
```

Consequently, WiX placed files from different subdirectories into the same installation directory, causing the filename collisions.

The corrected expression recognizes both Windows and Unix directory separators.

Step 3 — Rebuild the installer

Save your script, then execute:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

Step 4 — Check the results

If the build succeeds without `WIX1070` or `WIX1071` warnings, run:

```
Get-Item .\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi |
    Select-Object Name, Length, LastWriteTime
```

Then verify that WiX generated separate directories:

```
Select-String .\installer.generated.wxs -Pattern '<Directory Id=' |
    Select-Object -First 20
```

You should see directory definitions for the language resource folders and `wwwroot` structure.

Don't install the MSI just yet. Once the build is clean, we'll inspect the installation package and perform a controlled deployment without risking the existing TovarTech Asset Manager website or SQL Server database.

Send me the new build output, and we'll proceed to installation testing.

You said:

please provide the entire NEW code that i can replace the entire old one so i do not make a mistake

ChatGPT said:

Absolutely. Below is the **complete replacement for** `Build-MSI.ps1` for TovarTech Asset Manager 2.3. You do not need to find or edit individual lines.

This version fixes the Windows directory separator problem, preserves your published application's folder structure, removes stale publish files before rebuilding, and keeps your existing MSI version and UpgradeCode.

Step 1 — Back up and open your existing script

From your current project folder, run:

```
Copy-Item .\Build-MSI.ps1 .\Build-MSI.ps1.backup -Force
notepad .\Build-MSI.ps1
```

In Notepad, press **Ctrl+A** to select everything, delete it, and paste the complete code below.

Step 2 — Paste this entire new script

```
[CmdletBinding()]
param()

$ErrorActionPreference = 'Stop'

# ============================================================
# TovarTech Asset Manager Web
# MSI Build Script - Version 2.3.0
# Requires .NET 10 SDK and WiX Toolset
# ============================================================

$root = $PSScriptRoot

$project = Join-Path $root 'TovarTech.AssetManager.Web\TovarTech.AssetManager.Web.csproj'
$publish = Join-Path $root 'publish'
$output = Join-Path $root 'output'

$version = '2.3.0'
$upgradeCode = '5e0d9f2b-6b5a-4e71-9d61-9c6a2d86d7ac'

$wxsPath = Join-Path $root 'installer.generated.wxs'
$msiPath = Join-Path $output 'TovarTech_Asset_Manager_Web_2.3.0_x64.msi'

# ============================================================
# Helper functions
# ============================================================

function XmlEscape([string]$value) {
    return [System.Security.SecurityElement]::Escape($value)
}

# Keep the same GUID algorithm as the original script.
# This avoids unnecessary changes to component identity.
function StableGuid([string]$value) {

    $bytes = [System.Text.Encoding]::UTF8.GetBytes(
        $value.ToLowerInvariant()
    )

    $md5 = [System.Security.Cryptography.MD5]::Create()

    try {
        $hash = $md5.ComputeHash($bytes)
    }
    finally {
        $md5.Dispose()
    }

    return New-Object Guid (,$hash)
}

# Generate stable WiX identifiers from relative paths.
function StableId([string]$prefix, [string]$value) {
```

```
        $bytes = [System.Text.Encoding]::UTF8.GetBytes(
            $value.ToLowerInvariant()
        )

        $sha = [System.Security.Cryptography.SHA256]::Create()

        try {
            $hash = $sha.ComputeHash($bytes)
        }
        finally {
            $sha.Dispose()
        }

        $hex = [BitConverter]::ToString($hash).
            Replace('-', '').
            Substring(0, 24)

        return "$prefix$hex"
    }

    # ============================================================
    # Verify required tools
    # ============================================================

    Write-Host ''
    Write-Host '============================================' -ForegroundColor Cyan
    Write-Host ' TovarTech Asset Manager - MSI Build' -ForegroundColor Cyan
    Write-Host " Version $version" -ForegroundColor Cyan
    Write-Host '============================================' -ForegroundColor Cyan
    Write-Host ''

    if (-not (Test-Path $project)) {
        throw "Project file not found: $project"
    }

    if (-not (Get-Command dotnet -ErrorAction SilentlyContinue)) {
        throw 'The .NET SDK is not installed or is not in PATH.'
    }

    if (-not (Get-Command wix -ErrorAction SilentlyContinue)) {
        $env:PATH += ";$env:USERPROFILE\.dotnet\tools"
    }

    if (-not (Get-Command wix -ErrorAction SilentlyContinue)) {
        throw 'WiX Toolset was not found. Install WiX before continuing.'
    }

    # ============================================================
    # Clean and publish the application
    # ============================================================

    Write-Host 'Preparing publish and output folders...' -ForegroundColor Yellow

    if (Test-Path $publish) {
        Remove-Item -Path $publish -Recurse -Force
    }
```

```
New-Item -ItemType Directory -Path $publish -Force | Out-Null
New-Item -ItemType Directory -Path $output -Force | Out-Null

Write-Host 'Publishing the ASP.NET Core application...' -ForegroundColor Yellow

& dotnet publish $project `
    -c Release `
    -r win-x64 `
    --self-contained false `
    -o $publish

if ($LASTEXITCODE -ne 0) {
    throw 'dotnet publish failed.'
}

$files = @(
    Get-ChildItem -Path $publish -File -Recurse |
        Sort-Object FullName
)

if ($files.Count -eq 0) {
    throw 'Publish folder is empty.'
}

Write-Host "Published files: $($files.Count)" -ForegroundColor Green

# ============================================================
# Generate the WiX directory and component definitions
# ============================================================

Write-Host 'Generating WiX directory structure...' -ForegroundColor Yellow

$directoryIds = @{}
$directoryIds[''] = 'INSTALLFOLDER'

$directoryDefinitions =
    New-Object 'System.Collections.Generic.List[string]'

$componentDefinitions =
    New-Object 'System.Collections.Generic.List[string]'

$componentReferences =
    New-Object 'System.Collections.Generic.List[string]'

foreach ($file in $files) {

    # Convert the absolute path to a relative publish path.
    $relative = $file.FullName.Substring($publish.Length + 1)

    # CRITICAL FIX:
    # Normalize Windows backslashes to forward slashes.
    # This preserves cs/, de/, wwwroot/css/, etc.
    $relative = $relative.Replace('\', '/')

    $segments = $relative.Split('/')
```

```
        $parentId = 'INSTALLFOLDER'
        $currentPath = ''

        # Create directory definitions for each subfolder.
        for ($i = 0; $i -lt ($segments.Length - 1); $i++) {

            $segment = $segments[$i]

            if ($currentPath -eq '') {
                $currentPath = $segment
            }
            else {
                $currentPath += '/' + $segment
            }

            if (-not $directoryIds.ContainsKey($currentPath)) {

                $directoryId = StableId 'DIR_' $currentPath

                $directoryIds[$currentPath] = $directoryId

                $safeDirectoryName = XmlEscape $segment

                $directoryDefinitions.Add(
                    "<DirectoryRef Id=`"$parentId`">" +
                    "<Directory Id=`"$directoryId`" Name=`"$safeDirectoryName`" />" +
                    "</DirectoryRef>"
                )
            }

            $parentId = $directoryIds[$currentPath]
        }

        # Stable identifiers for each published file.
        $componentId = StableId 'CMP_' $relative
        $fileId = StableId 'FIL_' $relative

        # Preserve the original component GUID formula.
        $componentGuid = StableGuid (
            "TovarTechAssetManager2.3/" + $relative.Replace('/', '\')
        )

        $sourcePath = XmlEscape $file.FullName
        $fileName = XmlEscape $file.Name

        $componentDefinitions.Add(
            "<DirectoryRef Id=`"$parentId`">" +
            "<Component Id=`"$componentId`" Guid=`"$componentGuid`">" +
            "<File Id=`"$fileId`" Name=`"$fileName`" Source=`"$sourcePath`" KeyPath=`"yes`" />" +
            "</Component>" +
            "</DirectoryRef>"
        )

        $componentReferences.Add(
            "<ComponentRef Id=`"$componentId`" />"
        )
    }
```

```
$directoryXml = $directoryDefinitions -join "`r`n"
$componentXml = $componentDefinitions -join "`r`n"
$referenceXml = $componentReferences -join "`r`n"

# ============================================================
# Create the complete WiX installer definition
# ============================================================

$wxs = @"
<Wix xmlns="http://wixtoolset.org/schemas/v4/wxs">

  <Package
      Name="TovarTech Asset Manager Web"
      Manufacturer="TovarTech"
      Version="$version"
      UpgradeCode="$upgradeCode"
      Scope="perMachine">

    <MajorUpgrade
        DowngradeErrorMessage="A newer version is already installed." />

    <MediaTemplate EmbedCab="yes" />

    <StandardDirectory Id="ProgramFiles64Folder">
      <Directory
          Id="INSTALLFOLDER"
          Name="TovarTech Asset Manager Web" />
    </StandardDirectory>

$directoryXml

$componentXml

    <Feature
        Id="MainFeature"
        Title="TovarTech Asset Manager Web"
        Level="1">

$referenceXml

    </Feature>

  </Package>

</Wix>
"@

Set-Content -Path $wxsPath -Value $wxs -Encoding UTF8

Write-Host "Directory count: $($directoryIds.Count - 1)" -ForegroundColor Cyan
Write-Host "Component count: $($componentReferences.Count)" -ForegroundColor Cyan

# ============================================================
# Build the MSI
# ============================================================
```

```
Write-Host 'Building the Windows MSI installer...' -ForegroundColor Yellow

if (Test-Path $msiPath) {
    Remove-Item -Path $msiPath -Force
}

& wix build $wxsPath -arch x64 -out $msiPath

if ($LASTEXITCODE -ne 0) {
    throw 'WiX MSI build failed. Review the errors above.'
}

if (-not (Test-Path $msiPath)) {
    throw 'The MSI file was not created.'
}

# ============================================================
# Final build report
# ============================================================

Write-Host ''
Write-Host '============================================' -ForegroundColor Green
Write-Host ' MSI BUILD COMPLETED SUCCESSFULLY' -ForegroundColor Green
Write-Host '============================================' -ForegroundColor Green

Write-Host "MSI: $msiPath" -ForegroundColor Cyan

Get-Item $msiPath |
    Select-Object Name, Length, LastWriteTime |
    Format-Table -AutoSize

Write-Host ''
Write-Host 'IMPORTANT:' -ForegroundColor Yellow
Write-Host 'This MSI stages application files only.'
Write-Host 'IIS deployment requires a separate administrator step.'
Write-Host 'Your existing IIS website and SQL database have not been modified.'
```

Step 3 — Save and run

After pasting, save the file as `Build-MSI.ps1` (not `Build-MSI.ps1.txt`).

Then run:

```
powershell -ExecutionPolicy Bypass -File .\Build-MSI.ps1
```

If the build completes successfully, verify the output:

```
Get-Item .\output\TovarTech_Asset_Manager_Web_2.3.0_x64.msi |
    Select-Object Name, Length, LastWriteTime
```

Also check that the generated WiX definition contains subdirectories:

```
Select-String .\installer.generated.wxs -Pattern '<Directory Id=' |
    Select-Object -First 20
```

Expected result: The installer builds without the previous `WIX1070` and `WIX1071` short-filename collision warnings. If any warnings or errors remain, paste the exact output so we can address them.

Do not install the MSI yet. We'll verify its contents first, then plan a controlled deployment that preserves your existing IIS installation and SQL Server database.

You said:

Thsi is now working perfectly. My goal is to put everthing i need in github so that i can deploy this to other servers and make training videos for my youtube channel. Please provide a package that i can download that includes everything we need for this current version. it will include the build for the msi with step by step instructions in a read me file to package with it or that i can download from here and then upload with the build package, do not forget to tell me in the steps what else i need to install on my computer in the future to compile or build this installer.

Worked for 1m 19s

ChatGPT said:

I'll prepare a **TovarTech Asset Manager 2.3 GitHub Release Kit** that you can download, keep as your master source package, and use for future deployments and YouTube training demonstrations.

The package will include the application source, the corrected MSI build script, GitHub-ready files, and a step-by-step README covering the software required on a new Windows computer, building the installer, and deploying to another Windows Server.

One important distinction for your training videos: **building the MSI and deploying the ASP.NET Core application to IIS are separate operations**. The MSI stages the application files; the IIS website, SQL Server connection, and database still require configuration.

Your GitHub release package is ready

TovarTech Asset Manager 2.3 — GitHub Release Kit
ZIP archive · 22 files · Approximately 92 KB

Download the complete release kit ↗

The package contains the available Version 2.3 application source, corrected `Build-MSI.ps1`, updated package references for SQL Client 7.1.1 and PKCS 10.0.13, `.gitignore`, and two guides.

You can also download the instructions separately:

Step-by-step README ↗

Release checklist ↗

What you'll need on a future build computer

Software	Purpose
.NET 10 SDK (x64)	Compile and publish the application
WiX Toolset CLI	Generate the Windows MSI
Git for Windows	Manage and upload source to GitHub
Visual Studio Code	Edit and maintain the project (optional)

IIS + .NET 10 Hosting Bundle	Run the application on a target web server
SQL Server / SQL Server Express	Store application data

The README includes installation links, PowerShell commands, security checks, deployment guidance, and a suggested YouTube tutorial outline.

Important: The downloadable ZIP contains source and build instructions, not a precompiled MSI or database. It was assembled from your previously provided Version 2.3 source package and the corrections we made together. Your latest working Windows project may contain additional local changes, so compare the two before treating this as your final GitHub master. I verified the ZIP archive's integrity, but I haven't compiled this reconstructed package on Windows.

Recommended GitHub structure

```
TovarTech-Asset-Manager/
├── TovarTech.AssetManager.Web/
│   ├── Pages/
│   ├── wwwroot/
│   ├── Asset.cs
│   ├── AssetRepository.cs
│   ├── Program.cs
│   └── TovarTech.AssetManager.Web.csproj
├── Build-MSI.ps1
├── README.md
├── RELEASE-CHECKLIST.md
└── .gitignore
```

My recommendation is to keep the **source code in your GitHub repository** and publish each tested MSI as a downloadable **GitHub Release**. That gives viewers two options: learn how to compile the application themselves, or download a tested installer.

For your first public release, I recommend creating a separate lab deployment video demonstrating installation on Windows Server with IIS and SQL Server Express, using sample asset data rather than your production database.

README.md
Document

RELEASE-CHECKLIST.md
Document

You said:

when i try to download the complete release kit i get flagged as virus detected please provide a new virus free download package

Worked for 29s

ChatGPT said:

I've created a new **text-only TovarTech Asset Manager 2.3 package** designed to reduce the chance of antivirus warnings.

TovarTech Asset Manager 2.3 — Text-Only Package
ZIP · 21 files · Approximately 17 KB

Download the new package ↗

What I changed

- Removed compiled executables, DLLs, MSI files, and images.
- Included the application source code and GitHub documentation.
- Renamed `Build-MSI.ps1` to `Build-MSI.ps1.txt` for safer transfer.
- Added a new README explaining how to restore and build the project.
- Verified that the ZIP archive is structurally intact.

Important: I cannot guarantee that an antivirus product will classify any file as virus-free, and I haven't performed a Windows Defender scan. This package contains no compiled executable files.

After downloading

Extract the archive, rename `Build-MSI.ps1.txt` to `Build-MSI.ps1`, and copy your trusted `tovartech-logo.png` from your working project into the `wwwroot\images` folder.

The README explains how to install .NET 10 SDK and WiX, rebuild the MSI, prepare IIS, and upload the project to GitHub.

If Windows Defender still blocks this package, **do not disable your antivirus**. Send me the exact threat name displayed by Defender, and I'll help identify which file or behavior is triggering the detection.

You said:

Thank you I now have a back up can you provide a link to the entire chain of this chat from the point you suggested the 5 builds

ChatGPT said:

You're welcome! I'm glad you now have a backup of **TovarTech Asset Manager 2.3**.

You can preserve the entire development conversation, starting from when we discussed the five application builds, including the code, troubleshooting, MSI installer creation, and GitHub release preparation.

To get a link to the conversation:

1. Open the original **Application Ideas** conversation in your ChatGPT history.
2. Scroll to the point where we discussed the five application builds.
3. Click **Share** near the top of the conversation.
4. Select **Create link**, then **Copy link**.
5. Save that link with your GitHub project documentation.

Important: A ChatGPT shared link generally includes the conversation up to the point it was shared, rather than starting at a specific message. I cannot generate that share link directly from here.

For a permanent backup, I also recommend saving the conversation as a PDF using your browser's **Print → Save as PDF** feature.

That would give you both your application source code and a record of the development process for future TovarTech YouTube tutorials.